SVEUČILIŠTE JURJA DOBRILE U PULI
FAKULTET INFORMATIKE

Adis Spahić

Skladište podataka za analizu cestovnog prometa temeljeno na vlastitom sustavu prikupljanja

SEMINARSKI RAD

Pula, rujan, 2026. godine

SVEUČILIŠTE JURJA DOBRILE U PULI
FAKULTET INFORMATIKE

Adis Spahić

Skladište podataka za analizu cestovnog prometa temeljeno na vlastitom sustavu prikupljanja

SEMINARSKI RAD

JMBAG: 0303115252
Studijski smjer: Informatika
Kolegij: Skladišta i rudarenje podataka
Mentor: izv. prof. dr. sc. Goran Oreški

Pula, rujan, 2026. godine

IZJAVA O AKADEMSKOJ ČESTITOSTI

Ja, dolje potpisani Adis Spahić, ovime izjavljujem da je ovaj seminarski rad rezultat isključivo mojega vlastitog rada, da se temelji na mojim istraživanjima te da se oslanja na objavljenu literaturu kao što to pokazuju korištene bilješke i bibliografija. Izjavljujem da niti jedan dio seminarskog rada nije napisan na nedozvoljen način, odnosno da je prepisan iz kojega necitiranog rada, te da ikoji dio rada krši bilo čija autorska prava. Izjavljujem, također, da nijedan dio rada nije iskorišten za koji drugi rad pri bilo kojoj drugoj visokoškolskoj, znanstvenoj ili radnoj ustanovi.



Student

U Puli, rujan, 2026. godine

Sadržaj

1 Uvod **1**

2 Projektni zadatak **1**

2.1 Definicija problema i motivacija 1

2.2 Cilj i opis projektnog zadatka 2

2.3 Opseg i ograničenja projekta 2

2.4 Očekivani rezultati i doprinosi 2

2.5 Metodologija rada 3

3 Sustav za prikupljanje podataka **3**

3.1 Arhitektura sustava 4

3.2 Izvor videozapisa i mjerna lokacija 6

3.3 Obrada videotoka 6

3.4 Detekcija pokreta i odabir najboljeg kadra 7

3.4.1 Zona interesa i mjera zauzeća 8

3.4.2 Zaštite od lažnih detekcija 8

3.4.3 Odabir jednog kadra po prolasku 9

3.5 Izdvajanje uzorka i objava 9

3.6 Red poslova i pozadinska obrada 10

3.7 Anotacija vizijsko-jezičnim modelom 10

3.8 Obogaćivanje meteorološkim podacima 11

3.9 Kontrola kvalitete i ručna revizija 11

4 Skup podataka i eksploratorna analiza **12**

4.1 Struktura i atributi skupa podataka 12

4.2 Opseg i vremenska pokrivenost 13

4.3 Deskriptivna statistika 14

4.3.1 Sastav prometa 14

4.3.2 Mjere događaja 15

4.4 Kvaliteta podataka i nedostajuće vrijednosti 16

4.4.1 Zamjena nepouzdanih atributa vanjskim izvorom 17

5 Relacijski model podataka **17**

5.1 Priprema i podjela podataka 17

5.2 Analiza entiteta i poslovnih pravila 19

5.3 Konceptualni model 20

5.4 Prošireni model 22

5.5 Normalizacija i projektne odluke 24

5.5.1 Osvjetljenje pripada prolasku, ne satu 24

5.5.2 Nepoznata vrijednost je redak, a ne prazno polje 24

5.5.3 Ograničenja vrijednosti su zadržana 24

5.5.4 Šifrarnik ne sadrži sve vrijednosti 25

5.6 Implementacija u sustavu PostgreSQL 25

5.7 Uvoz podataka i validacija 26

6 Dimenzijski model podataka **27**

6.1 Postupak dizajna 27

6.2 Zrnatost tablice činjenica 28

6.3 Tablica činjenica 28

6.3.1 Mjere 28

6.3.2 Degenerirana dimenzija 29

6.4 Dimenzije i surogat ključevi 29

6.5 Hijerarhije 30

6.6 Napredne izvedbe dimenzijskih tablica 30

6.6.1 Sporo mijenjajuća dimenzija tipa 2 31

6.6.2 Kompozitna dimenzija 31

6.6.3 Heterogena dimenzija 32

6.6.4 Grupiranje kvantitativnih vrijednosti 32

6.7 Zvjezdasta shema 33

6.8 Prednosti zvjezdastog modela 35

7 ETL proces **36**

7.1 Arhitektura ETL rješenja 36

7.2 Faza izvlačenja (extract) 38

7.3 Faza transformacije (transform) 40
7.3.1 Transformacija dimenzijskih tablica 40
7.3.2 Transformacija tablice činjenica 42
7.4 Faza učitavanja (load) 45
7.5 Verifikacija učitanih podataka 46
7.6 Prednosti implementiranog ETL rješenja 48

8 OLAP analiza i vizualizacija **48**
8.1 Tableau kao OLAP alat 48
8.2 Ploča 1 — Pregled prometa: operacije ROLL-UP, DRILL-DOWN i SLICE 49
8.3 Ploča 2 — Ritam prometa i uvjeti: operacije DICE i PIVOT 51
8.4 Ploča 3 — Anomalije i kvaliteta podataka: operacije SLICE, PIVOT i DRILL-DOWN 54
8.5 Zbirni pregled OLAP operacija 57

9 Rasprava i ograničenja **59**
9.1 Ograničenja mjerne postavke 59
9.2 Pouzdanost anotacija vizijsko-jezičnog modela 59
9.3 Nejednolika pokrivenost uzorka 60
9.4 Primjena rezultata u produkcijskom sustavu 60

10 Zaključak **61**

Literatura **63**

Popis slika **64**

Popis tablica **65**

1 Uvod

Podaci o cestovnom prometu prikupljaju se danas gotovo posvuda, no najčešće završe kao puki broj vozila u nekom razdoblju. Takav podatak odgovara na pitanje koliko je vozila prošlo, ali ne i kakav je to promet bio — od čega se sastojao, kako se mijenjao kroz dan i je li na njega utjecalo vrijeme. Za odgovore te vrste potrebno je podatke organizirati tako da se mogu promatrati iz više kutova istodobno, a upravo je to zadaća skladišta podataka i dimenzijskog modeliranja.

Ovaj rad prolazi cijeli taj put, od snimke prometne kamere do interaktivne analitičke ploče. Za razliku od uobičajenog pristupa, u kojem se polazi od preuzetog skupa podataka, ovdje je skup izgrađen vlastitim sustavom za prikupljanje: kamera na ulazu u grad, prepoznavanje prolaska vozila i automatsko označavanje svake detekcije vizijsko-jezičnim modelom. Nastao je skup od 36.287 detekcija kroz dvanaest dana mjerenja, s dvadeset i sedam obilježja po detekciji.

Nad tim su skupom zatim provedeni svi koraci koje kolegij predviđa: eksploratorna analiza, relacijski model i njegova implementacija u bazi, dimenzijski model u obliku zvjezdaste sheme sa sedam dimenzija, ETL proces koji skladište puni iz dvaju izvora te analiza u alatu za vizualizaciju s demonstracijom OLAP operacija.

Rad je podijeljen u deset poglavlja. Poglavlje 2 definira projektni zadatak i metodologiju, a poglavlje 3 opisuje sustav kojim su podaci prikupljeni. Poglavlje 4 donosi eksploratornu analizu dobivenog skupa. Poglavlja 5 i 6 bave se relacijskim i dimenzijskim modelom, poglavlje 7 ETL procesom, a poglavlje 8 analizom i vizualizacijom. Poglavlje 9 raspravlja o ograničenjima, a poglavlje 10 donosi zaključak.

2 Projektni zadatak

2.1 Definicija problema i motivacija

Zadatak kolegija jest izgraditi skladište podataka i nad njim provesti analitičku obradu. Uobičajen je put za to preuzeti gotov skup podataka s nekog javnog repozitorija i nad njim proći korake modeliranja. Takav pristup ima jednu slabost: podaci dolaze već očišćeni i dokumentirani, pa se pitanja o njihovu nastanku i pouzdanosti nikada ne postave.

U ovom je radu odabran drugi put — skup podataka izgrađen je od nule, vlastitim sustavom za prikupljanje. Motivacija je dvojaka. Prvo, time se cijeli lanac podataka nalazi u istim rukama, od senzora do izvještaja, pa svako svojstvo skupa ima poznat uzrok. Drugo, postojeći produkcijski sustavi za brojanje prometa u pravilu bilježe koliko je vozila prošlo, ali ne omogućuju da se ta brojka promatra po dimenzijama, spušta niz hijerarhiju ili uspoređuje kroz vrijeme. Analitički sloj koji ovaj rad izgrađuje upravo je ono što takvim sustavima nedostaje.

2.2 Cilj i opis projektnog zadatka

Cilj je izgraditi potpun lanac obrade podataka o cestovnom prometu, od snimke kamere do interaktivne analitičke ploče. Lanac se sastoji od šest koraka:

1. prikupljanje podataka s prometne kamere i njihovo automatsko označavanje,
2. eksploratorna analiza dobivenog skupa i utvrđivanje njegove kvalitete,
3. izrada relacijskog modela i uvoz podataka u bazu,
4. izrada dimenzijskog modela u obliku zvjezdaste sheme,
5. punjenje skladišta ETL procesom iz dvaju izvora,
6. analiza i vizualizacija uz demonstraciju OLAP operacija.

Prvi korak nije dio zahtjeva kolegija nego preduvjet koji je trebalo stvoriti da bi ostali koraci uopće imali nad čime raditi.

2.3 Opseg i ograničenja projekta

Opseg je namjerno ograničen kako bi svaki korak lanca bio proveden do kraja, a ne samo naznačen. Mjerenje obuhvaća jednu kameru na jednoj lokaciji i razdoblje od dvanaest dana. Skladište ima jednu tablicu činjenica i sedam dimenzija, a analiza je izvedena u jednom alatu.

Izvan opsega ostaje sve što bi zahtijevalo dulje ili šire mjerenje: usporedba lokacija, sezonske razlike i praćenje trendova kroz godinu. Podrobniji popis ograničenja, zajedno s njihovim učinkom na zaključke, dan je u poglavlju 9.

2.4 Očekivani rezultati i doprinosi

Rad daje pet rezultata: vlastiti skup podataka o cestovnom prometu, relacijski model s pripadajućom bazom, dimenzijski model u obliku zvjezdaste sheme, ETL proces koji skladište puni iz dvaju izvora te tri analitičke ploče s demonstracijom OLAP operacija.

Doprinos u odnosu na uobičajenu izvedbu ovakvog zadatka leži u prvom od tih rezultata. Budući da je skup podataka vlastit, poznati su i njegovi nedostaci — koliko je koji atribut pouzdan, zašto neke vrijednosti nedostaju i kako su nastale one koje postoje. To je omogućilo dvije stvari koje s preuzetim skupom ne bi bile moguće: mjerenje točnosti pojedinih atributa nad ručno provjerenim uzorkom i vođenje promjene inačice modela za označavanje kao sporo mijenjajuće dimenzije.

2.5 Metodologija rada

Rad je izveden kroz pet kontrolnih točaka kolegija, redom, pri čemu svaka gradi na rezultatu prethodne. Tablica 1 povezuje kontrolne točke s poglavljima ovog rada i s programskom izvedbom.

Tablica 1: Kontrolne točke, pripadajuća poglavlja i programska izvedba

Kontrolna točka	**Poglavlje**	**Programska izvedba**
1 — odabir i analiza podataka	4	`01_eda_analysis.py`
2 — relacijski model	5	`02_preprocess_data.py`, `03_ddl_import.py`, `04_import_check.py`, `05_verify_import.sql`
3 — dimenzijski model	6	`generate_star_schema.py`
4 — punjenje skladišta	7	`main.py` te moduli `extract`, `transform` i `load`, `verify_etl.sql`
5 — analiza uz OLAP alat	8	`traffic_olap.twb`

Sustav za prikupljanje podataka, opisan u poglavlju 3, u toj tablici nema svoj redak jer nije dio nijedne kontrolne točke. On je nastao prije njih i predstavlja preduvjet cijelog rada.

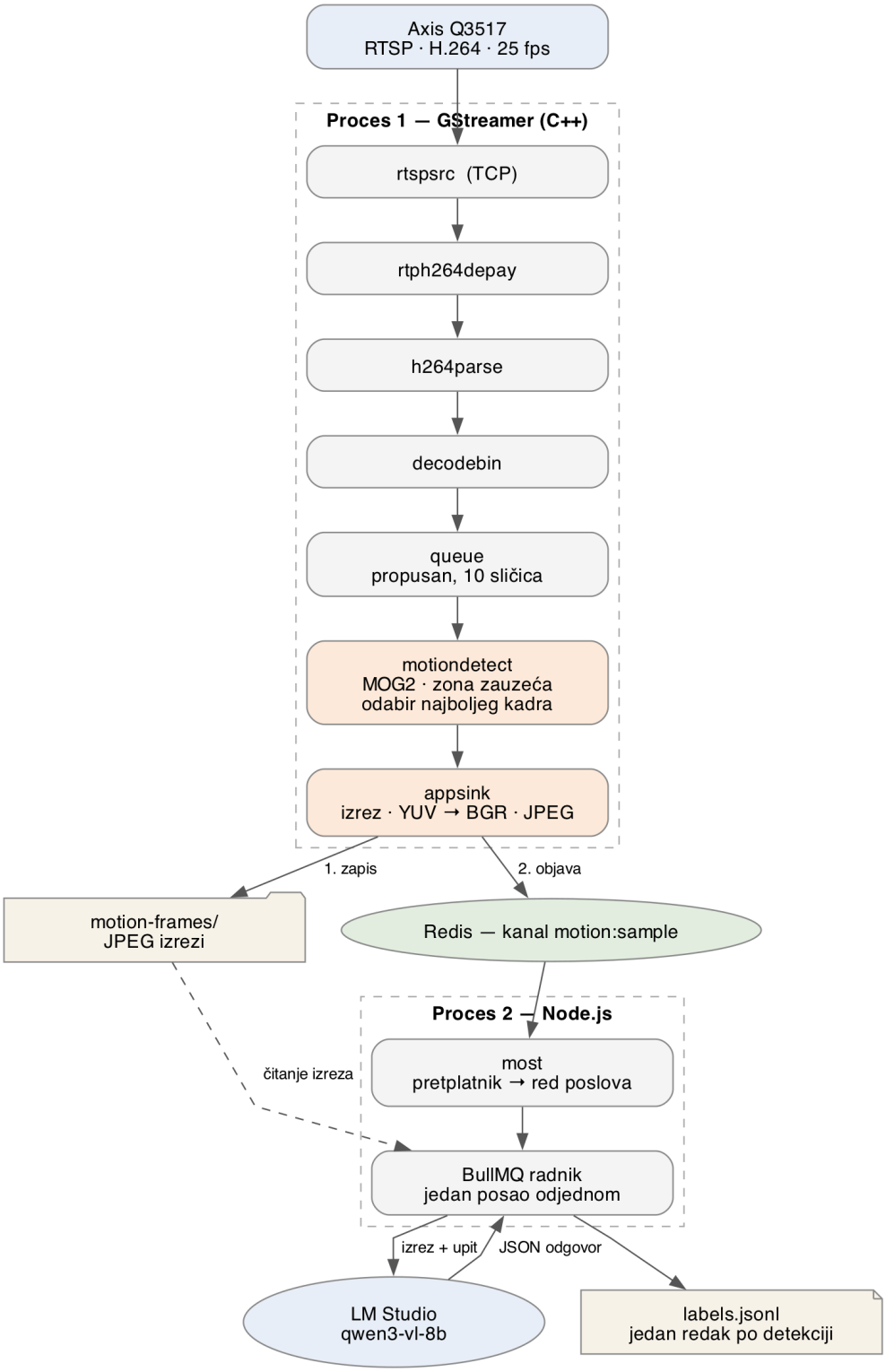
Za izradu su korišteni PostgreSQL kao sustav za upravljanje bazom podataka, Python sa SQLAlchemy za modeliranje i uvoz, Apache Spark za ETL proces te Tableau Desktop za analizu i vizualizaciju. Svaki je korak provjeren prije prelaska na sljedeći, a postupci provjere opisani su u pripadajućim poglavljima.

3 Sustav za prikupljanje podataka

Skup podataka na kojem se temelji ovaj rad nije preuzet s javne platforme, nego je izgrađen namjenski, za potrebe ovog projekta. Ta odluka određuje cijelo poglavlje: umjesto opisa tuđe datoteke slijedi opis mjernog postava, obrade videotoka, automatske anotacije i kontrole kvalitete. Poglavlje ne ulazi u implementacijske detalje na razini izvornog koda, nego objašnjava tijek podatka od trenutka kada vozilo uđe u vidno polje kamere do trenutka kada njegov opis postane redak u datoteci.

3.1 Arhitektura sustava

Sustav se sastoji od dva neovisna procesa povezana posrednikom poruka. Prvi proces napisan je u jeziku C++ i oslanja se na multimedijski okvir GStreamer [1]; njegova je zadaća primiti videotok s kamere, prepoznati prolazak vozila i na disk zapisati jedan isječak slike po prolasku. Drugi proces napisan je u okruženju Node.js i njegova je zadaća taj isječak poslati vizijsko-jezičnom modelu na anotaciju te strukturirani odgovor zapisati u datoteku. Između njih stoji Redis, koji ovdje ima ulogu posrednika poruka.



Slika 1: Arhitektura sustava za prikupljanje podataka

Podjela na dva procesa nije proizvoljna. Posao snimanja i posao anotacije imaju bitno različite vremenske zahtjeve: videotok pristiže neprekidno i ne smije se zaustaviti, dok jedan poziv vizijsko-jezičnog modela traje reda veličine nekoliko sekundi. Kada bi se oba posla odvijala u istom procesu, trajanje jednog zaključivanja izravno bi blokiralo primanje sljedećih sličica.

Iz istog razloga objava poruke prema Redisu izvedena je kao operacija bez čekanja na potvrdu. Izvor istine nije red poruka, nego mapa na disku u koju se zapisuju isječci: ako je posrednik nedostupan, snimanje se nastavlja neometano, a propuštene objave naknadno se nadoknađuju pregledom mape. Poruka se, uz to, objavljuje tek nakon što je zapis datoteke uspješno dovršen, pa drugi proces nikada ne dobije putanju do datoteke koja još nije potpuna.

3.2 Izvor videozapisa i mjerna lokacija

Podaci su prikupljeni s jedne nepomične kamere Axis Q3517 postavljene na ulazu u grad Novigrad, u Mlinskoj ulici 43. Kamera isporučuje videotok protokolom RTSP, kodiran standardom H.264, pri 25 sličica u sekundi. Njezine su koordinate (45,3160 N, 13,5627 E) kasnije iskorištene i kao ključ za dohvat meteoroloških podataka i kao geografska dimenzija u skladištu podataka.

Prijenos se odvija isključivo preko protokola TCP. To nije zadana postavka, nego posljedica mjerenja: prijenos preko UDP-a na istoj vezi gubio je približno 32 % paketa, pri čemu ono što stigne nisu cjelovite sličice nego njihovi fragmenti koje sljedeći element prosljeđuje kao da su ispravan video. Isti prijenos preko TCP-a na istoj vezi nije izgubio nijedan paket. Uz to je postavljen nadzor mrtve veze na pet sekundi, tako da prekinuti tunel brzo postane vidljiva pogreška, a ne tiho zaustavljanje.

Sustav se oslanja na jednu kameru i time na jednu mjernu lokaciju. To je svjesno ograničenje projekta i o njegovim se posljedicama raspravlja u poglavlju 9.

3.3 Obrada videotoka

GStreamer je okvir za izgradnju multimedijskih cjevovoda: obrada se opisuje kao niz povezanih elemenata kroz koje podaci teku od izvora prema odredištu. Okvir je pisan u jeziku C i danas je široko zastupljen u sustavima za računalni vid, između ostalog kao osnova Nvidijina alata DeepStream. Za ovaj projekt bitno je da uz same elemente donosi i rješenja za pregovaranje o formatima, upravljanje protokom i višedretvenost, što su svojstva bez kojih bi se prijem videotoka i njegova obrada morali pisati ispočetka.

Cjevovod ovog projekta sastavljen je većinom od gotovih elemenata okvira i jednog vlastitog elementa koji nosi poslovnu logiku. Tablica 2 navodi ih redom kojim podaci kroz njih prolaze.

Tablica 2: Elementi cjevovoda za prikupljanje podataka

Element	Uloga
`rtspsrc`	Uspostavlja i održava RTSP sesiju prema kameri te prima kodirani videotok u obliku RTP paketa.
`rtph264depay`	Izdvaja H.264 jedinice iz RTP paketa, odnosno uklanja transportni omotač.
`h264parse`	Raščlanjuje H.264 tok i dopunjuje ga zaglavljima koja dekoder očekuje.
`decodebin`	Sam odabire i konfigurira dekoder te isporučuje dekodirane sličice.
`queue`	Međuspremnik koji razdvaja dekodiranje od detekcije i propušta ih u zasebnim dretvama.
`motiondetect`	Vlastiti element: prepoznaje prolazak vozila i bira jedan kadar po prolasku.
`appsink`	Izrezuje odabrani kadar, pretvara ga u boju, zapisuje kao JPEG i objavljuje poruku.

Položaj i postavke elementa `queue` zaslužuju posebno objašnjenje jer su izravna posljedica toga što je detekcija najskuplji korak cjevovoda. Međuspremnik je smješten neposredno prije detekcije i ograničen na deset sličica. Ključna je postavka da je propusan (*leaky*) u smjeru toka: kada se napuni, odbacuje najstariju sličicu umjesto da zaustavi dotok. Bez toga bi se pritisak širio unatrag — međuspremnik bi se napunio, dekodiranje bi stalo, a zastoj bi na kraju stigao do same RTSP sesije. Odbacivanje pojedine sličice pritom je bezopasno: prolazak vozila obuhvaća niz uzastopnih sličica — izmjereni medijan trajanja događaja iznosi 2,1 sekundu — pa gubitak jedne ne znači gubitak vozila.

Vrijedi napomenuti i što u cjevovodu namjerno *ne* postoji. Nigdje prije odredišta nema pretvorbe boje. Dekodirane sličice ostaju u izvornom YUV zapisu, u kojem je svjetlina odvojena od boje, a detekcija čita samo komponentu svjetline. Pretvorba cijele sličice u boju za svaku primljenu sliku bila bi najskuplja pojedinačna operacija u cjevovodu, a boja je potrebna samo na malenom izrezu koji na kraju odlazi modelu.

3.4 Detekcija pokreta i odabir najboljeg kadra

Vlastiti element `motiondetect` obavlja dva posla: odlučuje je li na sličici prisutno vozilo i, kada jest, bira jedan kadar koji će ga predstavljati.

3.4.1 Zona interesa i mjera zauzeća

Detekcija se temelji na oduzimanju pozadine algoritmom MOG2 iz biblioteke OpenCV. Algoritam gradi statistički model svakog piksela i sve što od njega odudara označava kao prednji plan. Nad tom se maskom mjeri *zauzeće*: udio piksela prednjeg plana unutar zone interesa. Ako je taj udio veći od praga (zadano 0,015), smatra se da je u zoni vozilo.

Bitno je da se ne mjeri apsolutan broj piksela nego udio, i da se ne traže obrisi ni okviri oko objekata. Zona interesa dimenzionirana je tako da u nju stane jedno vozilo, pa automobil zauzima velik dio zone, a šum tek neznatan; zona time radi slično induktivnoj petlji ugrađenoj u asfalt pred semaforom.

Sustav koristi dva pravokutnika, a ne jedan. Prvi, *zona odluke*, određuje kada se smatra da je vozilo prisutno. Drugi, *zona izreza*, ujedno je radno područje koje algoritam obrađuje i točan okvir svakog uzorka koji izlazi iz sustava. Time detekcija vidi upravo ono što će vidjeti i model, a budući da je okvir izreza nepromjenjiv i dovoljno velik da primi najveće očekivano vozilo, izrez s pola automobila nije moguć po konstrukciji.

Postavljanje zone strateški je odabrano na dijelu ceste kojim vozila prolaze. Razlog je što prag zauzeća sam po sebi ne razlikuje vozilo od pješaka: zbog perspektive pješak bliže kameri zauzima više piksela nego vozilo dalje od nje, pa je geometrijsko ograničavanje zone pouzdanije od bilo kakvog praga. Ta je pouka naslijeđena iz ranijeg sustava u kojem je zona bila izmjerena na razlučivosti 2560×1440, a kamera je poslije prebačena na 1280×720; neprilagođena zona tiho je skliznula na pješačku površinu, pa je detekcija cijeli dan reagirala na pješake, a nikad na promet. Ništa nije prijavilo pogrešku — rezultati su jednostavno bili krivi.

3.4.2 Zaštite od lažnih detekcija

Uz prag zauzeća element sadrži još tri mehanizma. Prvi je odbacivanje blještanja: ako srednja svjetlina piksela prednjeg plana prijeđe vrijednost 230, riječ je o snopu svjetla ili odsjaju na asfaltu, a ne o vozilu, i detekcija se poništava. Drugi je zaštita od promjene osvjetljenja: ako zauzeće prijeđe 60 % zone i takvo stanje potraje, nije riječ o vozilu nego o promjeni ekspozicije, pa se model pozadine gradi ispočetka. Treći je zagrijavanje modela, tijekom kojeg se prvih stotinu sličica koristi isključivo za izgradnju modela pozadine.

Vrijednosti parametara nisu pretpostavljene nego izmjerene na stvarnom prometu. Prag odudaranja modela MOG2 spušten je s početnih 16 na 8 nakon što je mjerenje pokazalo da pri višoj vrijednosti maska pokriva samo dijelove vozila koji jako odudaraju od pozadine: stvarni objekti stizali su kao krhotine s masama između 622 i 730 piksela, što se nije razlikovalo od šuma. Pri vrijednosti 8 isti je promet dao mase između 4611 i 39105 piksela, dakle razmak prema šumu postao je veći, a ne manji.

3.4.3 Odabir jednog kadra po prolasku

Kamera je konfigurirana na 25 sličica u sekundi, a prolazak vozila kroz zonu traje — mjereno nad cijelim skupom — medijalno 2,1 sekundu, uz interkvartilni raspon od 1,4 do 3,0 sekunde. Bez dodatne logike jedno bi vozilo pokrenulo detekciju na nizu uzastopnih sličica, što bi dalo isto toliko redaka u skupu podataka i isto toliko poziva modela — sve za isto vozilo.

Element to rješava malim automatom stanja. Prva sličica s detekcijom otvara *događaj*. Dok je događaj otvoren, element pamti najbolji kandidatni kadar i uspoređuje ga sa svakom novom sličicom: bolji je onaj s većim zauzećem, dakle onaj na kojem vozilo zauzima više piksela, što se u praksi poklapa s trenutkom kada je vozilo najbliže sredini zone. Događaj se zatvara na jedan od tri načina: kada nakon zadnje detekcije protekne razdoblje mirovanja (zadano 240 ms), kada se dosegne najdulje dopušteno trajanje događaja, ili kada završi videotok. Tek se pri zatvaranju prema odredištu šalje točno jedan spremnik — pobjednički kadar.

Uz taj se kadar prilaže i zapis o događaju: redni broj `motion_id`, okvir izreza, trajanje događaja, vršno zauzeće u pikselima i razlog zatvaranja. Trajanje se mjeri od prve sličice događaja do sličice na kojoj je zatvoren, pa uključuje i razdoblje mirovanja — ono je, dakle, gornja ocjena stvarnog prolaska, a ne njegova točna duljina. Trajanje događaja i vršno zauzeće kasnije postaju mjere `dur_ms` i `peak_mass` u tablici činjenica, gdje služe kao posredni pokazatelji zadržavanja u zoni i veličine vozila.

3.5 Izdvajanje uzorka i objava

Posljednji element cjevovoda, `appsink`, prima pobjednički kadar i pretvara ga u konačni uzorak u tri koraka. Najprije ga izrezuje na okvir zabilježen u priloženom zapisu, potom taj izrez — i samo njega — pretvara iz zapisa YUV u zapis BGR, i naposljetku ga zapisuje na disk kao JPEG imenovan rednim brojem događaja i vremenskom oznakom.

Oba koraka služe kvaliteti zaključivanja. Izrez uklanja sve što modelu nije od koristi: okolne zgrade, stabla, parkirana vozila i pješake izvan zone. Pretvorba u boju nužna je jer se boja vozila i njegove oznake bez nje ne mogu očitati. Budući da se pretvara samo izrez, a ne cijela sličica, trošak je zanemariv.

Nakon uspješnog zapisa poziva se Redis klijent i na kanalu objavljuje poruka. Poruka ne sadrži sliku nego njezine metapodatke: verziju zapisa, redni broj događaja, putanju do datoteke, vremensku oznaku u ISO formatu, razlog zatvaranja događaja, vršno zauzeće, trajanje, dimenzije izreza, oznaku kamere i položaj kadra na vremenskoj osi videotoka. Time zadaća prvog procesa završava.

3.6 Red poslova i pozadinska obrada

Na drugoj strani kanala nalazi se pretplatnik napisan u okruženju Node.js. Njegova je jedina uloga posrednička: preuzima poruku s kanala i pretvara je u posao u redu poslova kojim upravlja biblioteka BullMQ. Taj je međukorak potreban zato što je proizvođač poruka napisan u jeziku C: posao biblioteke BullMQ nije jedna vrijednost u Redisu nego skup ključeva koji se održavaju bibliotečnim skriptama, pa ga vanjski proces ne može ispravno stvoriti. Objavom obične JSON poruke i prijevodom na strani Node.js-a zadržava se jednostavnost na strani snimanja, a sve nizvodno ostaje uobičajeni BullMQ.

Prilikom stvaranja posla identifikator se izvodi iz rednog broja događaja. Time se postiže idempotentnost: ako je isti događaj ikada objavljen dvaput — primjerice pri naknadnoj obradi — biblioteka zadržava prvi posao, a drugi tiho odbacuje. Posao se pri neuspjehu ponavlja do tri puta s eksponencijalno rastućim odmakom.

Radnik obrađuje poslove jedan po jedan. To nije ograničenje izvedbe nego namjerna postavka: model odgovara serijski, pa bi veći broj istodobnih poslova samo premjestio čekanje iz reda poslova u sam poslužitelj modela, uz lošije vrijeme odziva po poslu.

3.7 Anotacija vizijsko-jezičnim modelom

Anotaciju obavlja vizijsko-jezični model `qwen3-vl-8b` [2], pokrenut lokalno u alatu LM Studio. Lokalno izvođenje bilo je preduvjet projekta: skup podataka sadrži preko trideset i pet tisuća izrezaka, pa bi anotacija preko naplatnog vanjskog servisa bila neprovediva.

Prije poziva izrez se dodatno obrađuje. Na sliku se ucrtava crveni pravokutnik koji označava zonu praćenja, a uputa modelu izričito traži da prijavi samo subjekte koji tu zonu barem dijelom dodiruju. Time se rješava problem koji se izrezom ne može riješiti do kraja: i unutar izreza mogu se naći vozila dublje u sceni ili parkirana uz rub, a bez vizualne oznake model nema način razlikovati ih od vozila koje zapravo prolazi.

Poziv se izvodi sučeljem kompatibilnim s OpenAI-jevim, uz temperaturu postavljenu na nulu radi ponovljivosti i uz strogu shemu odgovora. Shema je bitna jer model ne vraća slobodan tekst nego strukturu čija su polja unaprijed određena, s nabrojanim dopuštenim vrijednostima. Odgovor sadrži opis scene — vremenske prilike, osvjetljenje i stanje kolnika — te popis objekata, gdje svaki objekt nosi klasu, boju, tip karoserije, marku, model, dizajnersko razdoblje, smjer okrenutosti, upaljena svjetla, poslovne oznake, posebnu namjenu, oznaku odsječenosti i vidljivost registarske pločice.

Vrijedi istaknuti jednu odluku u oblikovanju sheme. Klasa `pedestrian` namjerno je zadržana među dopuštenim vrijednostima iako pješaci nisu predmet ovog skupa podataka. Razlog je što bi njezino uklanjanje ostavilo model bez istinitog odgovora kada u zoni doista stoji pješak, pa bi bio prisiljen odabrati neku vozilnu klasu. Pješaci i prazni kadrovi umjesto toga se filtriraju pri zapisu: bilježe se u dnevnik, ali ne ulaze u skup podataka.

Zapis je uređen tako da jedan redak odgovara jednoj detekciji, a ne jednoj slici. Kadar

može sadržavati više subjekata — automobil i motocikl mogu dijeliti isti prolazak — pa svaki objekt dobiva vlastiti identifikator sastavljen od rednog broja događaja i rednog broja objekta unutar odgovora. Polja koja opisuju scenu ponavljaju se u svakom retku istog kadra. Datoteka `labels.jsonl` zapisuje se uz same slike, tako da mapa ostaje cjelovit skup podataka.

3.8 Obogaćivanje meteorološkim podacima

Skup podataka dopunjen je drugim izvorom: satnom poviješću servisa Open-Meteo [3] za koordinate kamere i razdoblje prikupljanja. Dohvat je izveden zasebnom skriptom, a rezultat se čuva i kao samostalna datoteka, ne samo spojen u konačni skup.

Razlog nije ukrasan. Mjerenje je pokazalo da su modelovi stupci o vremenskim prilikama i osvjetljenju neupotrebljivi: u sirovom izlazu modela vrijednost `sunny` pojavljuje se u 99,3 % redaka, a vrijednost `day` u 95,6 % redaka snimljenih u jedan sat ujutro. Model, dakle, opisuje scenu koju vidi, ali ne razlikuje pouzdano ono što se s izreza ne može očitati. Vanjski izvor daje stvarno izmjerene vrijednosti po satu, a vremena izlaska i zalaska sunca daju egzaktno osvjetljenje umjesto procjene sa slike.

3.9 Kontrola kvalitete i ručna revizija

Automatska anotacija skupa podataka nema vrijednosti ako se njezina točnost ne izmjeri. Za tu je svrhu izdvojen slučajni uzorak od 150 detekcija koje su ručno označene *naslijepo*, dakle bez uvida u modelov odgovor, a tek je potom napravljena usporedba. Rezultati su prikazani u tablici 3.

Tablica 3: Izmjerena točnost anotacije na slijepom uzorku od 150 detekcija

Polje	Ocijenjeno	Točno	Točnost
Klasa vozila	147	119	81 %
Marka	54	47	87 %
Model	24	16	67 %
Boja	139	92	66 %
Tip karoserije	96	52	54 %
Smjer okrenutosti	147	58	39 %

Mjerenje je otkrilo tri različite vrste pogreške. Prva je pogreška prosuđivanja na granici kategorija: najčešće zamjene kod klase su `van` umjesto `car` (deset puta) i `scooter` umjesto `motorcycle` (šest puta), a kod boje `silver` umjesto `gray` (trinaest puta). Takve su zamjene razumljive i ne dovode u pitanje upotrebljivost polja.

Druga je suzdržanost. Marka je ocijenjena samo na 54 detekcije jer se model u ostalima

izjasnio s `unknown`. Točnost od 87 % odnosi se, dakle, na slučajeve u kojima je model tvrdio da nešto prepoznaje — a upravo je takvo ponašanje traženo uputom, koja izričito zabranjuje nagađanje.

Treća je sustavna pogreška, i ona je najvažniji nalaz mjerenja. Smjer okrenutosti točan je u samo 39 % slučajeva, ali raspodjela pogrešaka nije slučajna: model je 88 puta odgovorio `left` kada je istina bila `right`, a samo jednom obrnuto. Riječ je o dosljednoj pristranosti, a ne o šumu.

Taj je nalaz izravno odredio opseg naknadne revizije. Polje smjera — koje u skladištu podataka postaje atribut `direction` u dimenziji `dim_observation` — podvrgnuto je sustavnom pregledu nad cijelim skupom, a uz njega i polja posebne namjene i preostalih klasa. Revizija je provedena u dva prolaza, nad povijesnim i nad novijim dijelom skupa, uz provjeru cjelovitosti nakon svakog: broj presuda po polju mora odgovarati broju redaka. Iz skupa su uklonjeni i duplikati utvrđeni usporedbom sažetaka slika.

Zbog toga je smjer kretanja polje s najviše uloženog rada u cijelom skupu podataka, a njegovi nalazi u poglavlju 8 počivaju na provjerenim, a ne na izvornim modelovim vrijednostima.

4 Skup podataka i eksploratorna analiza

Skup podataka opisan u ovom poglavlju izravan je rezultat sustava iz poglavlja 3. Analiza koja slijedi obavljena je prije bilo kakvog modeliranja: njezina je svrha utvrditi što skup zapravo sadrži, koliko je pouzdan i koje transformacije treba obaviti prije nego što uđe u bazu.

Poglavlje odgovara prvoj kontrolnoj točki kolegija, a eksploratorna analiza provedena je skriptom `01_eda_analysis.py`.

4.1 Struktura i atributi skupa podataka

Skup je izvezen u dva istovjetna oblika — CSV i JSONL — i sadrži 27 atributa. Jedan redak odgovara jednoj detekciji vozila, a ne jednoj snimljenoj slici, pa jedan kadar može dati više redaka. Atributi se prirodno dijele u pet skupina, prikazanih u tablici 4.

Tablica 4: Atributi skupa podataka po skupinama

Skupina	**Atributi**
Identifikacija	`inference_id`, `motion_id`, `ts`, `file`, `camera_id`
Mjere događaja	`cause`, `peak_mass`, `dur_ms`, `pts_ms`
Provenijencija	`schema_v`
Vanjski izvor	`temp_c`, `precip_mm`, `weather_code`, `lighting`
Opis vozila	`class`, `color`, `body`, `make`, `model`, `era`
Opis promatranja	`direction`, `road_condition`, `headlights_on`, `commercial_markings`, `special_use`, `truncated`, `plate_visible`

Podjela nije samo opisna — ona već nagovješćuje dimenzijski model. Skupina koja opisuje vozilo govori o *čemu* se radi i kasnije postaje dimenzija vozila; skupina koja opisuje promatranje govori *kako* je vozilo viđeno i postaje zasebna dimenzija; mjere događaja su jedine brojčane veličine koje se smisleno zbrajaju i prosječuju, pa postaju mjere u tablici činjenica.

Atribut `schema_v` zaslužuje poseban spomen jer ne opisuje ni vozilo ni promatranje, nego *postupak* kojim je redak nastao. Njegova vrijednost govori kojom je verzijom upute i sheme model odgovorio, i upravo on kasnije omogućuje praćenje povijesti verzija u skladištu podataka.

4.2 Opseg i vremenska pokrivenost

Skup sadrži **36.287 detekcija** nastalih iz **35.360 zabilježenih prolazaka**. Razlika između te dvije brojke posljedica je toga što jedan prolazak može sadržavati više subjekata: automobil i motocikl koji istodobno uđu u zonu dijele isti `motion_id`, ali dobivaju zasebne retke.

Prikupljanje je trajalo od 20. kolovoza 2026. u 21:26 do 31. kolovoza 2026. u 06:58, dakle punih deset dana uz dva djelomična na rubovima. Tablica 5 pokazuje dnevni obujam.

Tablica 5: Broj detekcija po danima

Datum	Detekcija	Datum	Detekcija
20. 08.	300*	26. 08.	3.498
21. 08.	3.363	27. 08.	3.743
22. 08.	3.653	28. 08.	3.933
23. 08.	3.020	29. 08.	3.946
24. 08.	3.538	30. 08.	3.787
25. 08.	3.267	31. 08.	239*

*djelomičan dan

Deset punih dana kreće se u uskom rasponu od 3.020 do 3.946 detekcija dnevno, što govori da je sustav radio stabilno i bez prekida. Dva rubna dana — prvi obuhvaća samo večernje sate, posljednji samo jutarnje — moraju se isključiti iz svake usporedbe dnevnih količina, a u vizualizaciji su zadržani jer bi njihovo uklanjanje skratilo vremenski raspon.

Skup nije jednak sirovom izlazu sustava. Prije izvoza primijenjene su presude ručne revizije opisane u odjeljku 3.9, čime je iz 37.587 autoritativnih redaka uklonjeno 1.300 redaka, odnosno 3,5 %. Uklonjeni su vagoni turističkog vlaka (533 retka, jer jedan vlak treba brojati kao jedno vozilo), duplikati kadra utvrđeni usporedbom sažetaka (277), prividne detekcije bez vozila (248), objekti koji nisu vozila (209) i neodredive vrijednosti (33). Istim je postupkom ispravljena klasa 551 retka, od čega je 401 prepoznat kao turistički vlak.

4.3 Deskriptivna statistika

4.3.1 Sastav prometa

Raspodjela po klasama vozila izrazito je neujednačena, što je za gradski promet očekivano. Osobni automobili čine 53,3 % svih detekcija, a zajedno s dostavnim vozilima 72,0 %. Dvokotači — bicikli, romobili i motocikli — zajedno čine 26,7 %, što je za priobalni gradić u kolovozu razmjerno visok udio.

Tablica 6: Raspodjela detekcija po klasi vozila

Klasa	Detekcija	Udio
`car`	19.349	53,3 %
`van`	6.794	18,7 %
`bicycle`	4.157	11,5 %
`scooter`	3.213	8,9 %
`motorcycle`	2.312	6,4 %
`tourist_train`	401	1,1 %
`truck`	51	0,1 %
`bus`	10	0,0 %
Ukupno	**36.287**	**100 %**

Kamioni i autobusi pojavljuju se u zanemarivom broju. To nije nedostatak prikupljanja nego svojstvo lokacije: ulaz u grad kojim teški teretni promet ne prolazi. Te dvije klase zadržane su u skupu radi potpunosti, ali svaki zaključak izveden nad njima počiva na desecima, a ne tisućama opažanja.

Raspodjela smjera kretanja izrazito je asimetrična: 33.867 detekcija (93,3 %) ima smjer `right`, a 2.182 (6,0 %) smjer `left`. Preostalih 238 redaka nosi vrijednost `unknown` ili neku od rijetkih vrijednosti okrenutosti prema kameri. Ta je asimetrija očekivana — kamera gleda cestu s jednim prevladavajućim smjerom — a manjinski smjer upravo je zbog svoje rijetkosti analitički zanimljiv, o čemu se raspravlja u poglavlju 8.

Vozila s posebnom namjenom rijetka su: 1.399 dostavnih (3,9 %), 90 taksija (0,2 %) i 72 vozila hitnih službi (0,2 %); preostalih 34.726 redaka nema posebnu namjenu.

4.3.2 Mjere događaja

Dvije brojčane mjere opisuju sam prolazak. Trajanje događaja `dur_ms` kreće se od 280 do 11.927 milisekundi, s medijanom 2.079 i prosjekom 2.183. Vršno zauzeće `peak_mass` kreće se od 1.354 do 80.164 piksela, s medijanom 8.801 i prosjekom 11.458. Obje razdiobe imaju dug desni rep, što je vidljivo iz odnosa medijana i prosjeka.

Tablica 7: Razdioba mjera događaja

Mjera	min	Q1	medijan	Q3	P95	max
`dur_ms`	280	1.439	2.079	3.039	3.279	11.927
`peak_mass`	1.354	5.442	8.801	14.884	29.942	80.164

Nijedna od tih mjera nije izravno fizikalna veličina. Trajanje događaja mjeri zadržavanje u zoni i tek je posredan pokazatelj brzine, jer duljina putanje kroz zonu ovisi o traku i

smjeru kretanja. Vršno zauzeće mjeri broj piksela i tek je posredan pokazatelj veličine vozila, jer je uvjetovano i udaljenošću od kamere. Obje se u analizi koriste kao takve — kao pokazatelji, a ne kao mjerenja — i tako su i tumačene.

4.4 Kvaliteta podataka i nedostajuće vrijednosti

Skup ne sadrži nijednu neočekivano praznu vrijednost, ali sadrži znatan udio vrijednosti `unknown`. Razlika je bitna: prazna vrijednost znači da podatak nedostaje, a `unknown` je odgovor modela da prepoznavanje nije bilo moguće. Tablica 8 prikazuje udjele.

Tablica 8: Udio nepoznatih i praznih vrijednosti po atributu

Atribut	Vrijednost	Redaka	Udio
`model`	`unknown`	26.032	71,7 %
`make`	`unknown`	25.970	71,6 %
`era`	prazno	14.254	39,3 %
`body`	prazno	10.527	29,0 %
`era`	`unknown`	6.190	17,1 %
`color`	`unknown`	4.512	12,4 %
`direction`	`unknown`	213	0,6 %

Sva tri velika udjela imaju objašnjenje u načinu nastanka podataka i nijedan nije pogreška.

Visok udio nepoznate marke i modela izravna je posljedica upute koja modelu izričito zabranjuje nagađanje i traži odgovor `unknown` kada značka nije čitljiva. Mjerenje na slijepom uzorku iz odjeljka 3.9 pokazalo je da je takva suzdržanost opravdana: ondje gdje se model izjasnio, marka je bila točna u 87 % slučajeva. Skup, dakle, sadrži manje podataka o marki, ali ono što sadrži pouzdanije je nego da je model bio prisiljen pogađati.

Prazna vrijednost tipa karoserije nije nedostatak nego pravilo: uputa traži tip karoserije isključivo za osobne automobile i dostavna vozila, a za sve ostale klase očekuje praznu vrijednost. Provjera to potvrđuje — među 26.143 automobila i dostavnih vozila tip karoserije nedostaje u samo 383 retka, odnosno 1,5 %.

Prazno dizajnersko razdoblje ima potpuno drugačiji uzrok i to je najvažniji nalaz ovog odjeljka. Atribut `era` uveden je tek u četvrtoj verziji sheme; svih 14.254 redaka kojima nedostaje potječe iz treće verzije, gdje taj atribut nije postojao. Broj se poklapa točno: skup sadrži 14.254 retka verzije 3 i 22.033 retka verzije 4. Nije, dakle, riječ o podatku koji se izgubio, nego o podatku koji u trenutku nastanka retka nije ni postojao. Takva promjena strukture kroz vrijeme upravo je situacija koju dimenzijsko modeliranje rješava praćenjem povijesti verzija, o čemu se govori u poglavlju 6.

4.4.1 Zamjena nepouzdanih atributa vanjskim izvorom

Dva atributa koja opisuju scenu — vremenske prilike i osvjetljenje — pokazala su se nepouzdanima i zamijenjena su vrijednostima iz vanjskog izvora, kako je obrazloženo u odjeljku 3.8. Treći atribut te vrste, stanje kolnika, zadržan je onakav kakvim ga je model prijavio jer se odnosi na ono što se sa slike doista može očitati. Razmjeri intervencije nad preostala dva vidljivi su tek iz usporedbe.

Model je vremenske prilike opisao kao `sunny` u 36.100 od 36.287 redaka, dakle u 99,5 %, uz svega 187 redaka s kišom. Izmjerene vrijednosti daju bitno drugačiju sliku: vedro 17.041, pretežno vedro 7.373, rosulja 5.317, djelomično oblačno 3.153, kiša 2.026 i oblačno 1.377 redaka. Ukupno 7.343 detekcije dogodile su se uz mjerljive oborine — otprilike petina skupa — dok ih je model prepoznao kao takve tek 187.

Osvjetljenje je ispravljeno u 8.050 redaka, odnosno u 22,2 % skupa. Model je noć prepoznao u 1.421 retku, a izračun iz vremena izlaska i zalaska sunca daje 6.557. Uzrok je razumljiv — ulaz u grad je noću osvijetljen javnom rasvjetom, pa scena modelu izgleda kao dan — ali posljedica bi bila ozbiljna: analiza po dobu dana temeljila bi se na atributu koji noć propušta u više od tri četvrtine slučajeva.

Spajanje s vanjskim izvorom provedeno je bez gubitka: satna povijest pokriva svih 288 sati razdoblja i nijedan redak nije ostao bez pripadajućeg meteorološkog zapisa.

5 Relacijski model podataka

5.1 Priprema i podjela podataka

Prije nego što skup uđe u bazu, nad njim su obavljene tri pripremne radnje. Provodi ih skripta `02_preprocess_data.py`, prva u nizu skripti druge kontrolne točke.

Kanonikalizacija marke. Sirovi skup sadrži 87 različitih vrijednosti atributa `make`, ali to nije 87 proizvođača. Postupak ih svodi u dva koraka. Prvi je normalizacija zapisa: uklanjanje suvišnih razmaka, svođenje na mala slova i uklanjanje dijakritičkih znakova, čime se 87 vrijednosti svodi na 61. Preostalih 26 razlika bile su, dakle, samo različiti načini pisanja istog imena. Drugi je korak popis iznimaka: kratice i nedosljedni zapisi preslikavaju se na kanonski oblik (`vw` i `vw caddy` na `Volkswagen`, `doblo` na `Fiat`), a vrijednosti koje uopće nisu proizvođači nego opisi vozila (`suv`, `minivan`, `atv`) prebacuju se u `unknown`. Nakon toga ostaje 51 vrijednost, odnosno 50 stvarnih proizvođača uz `unknown`.

```
MAKE_ALIASES = {
    "vw": "volkswagen",
    "vw caddy": "volkswagen",
    "range rover evoque": "range rover",
    "doblo": "fiat",
```

```
    "infinite": "infiniti",
}

MAKE_NON_MANUFACTURER = {
    "suv": "unknown",
    "minivan": "unknown",
    "atv": "unknown",
    "candy": "unknown",
    "stummers": "unknown",
}

MAKE_DISPLAY_EXCEPTIONS = {
    "bmw": "BMW", "mg": "MG", "ktm": "KTM",
    "gmc": "GMC", "ram": "RAM",
}

def make_key(value):
    if pd.isna(value):
        return value
    value = str(value).strip()
    value = " ".join(value.split())
    value = unicodedata.normalize("NFD", value)
    return "".join(ch for ch in value
                   if unicodedata.category(ch) != "Mn").lower()

def make_display(key):
    if pd.isna(key):
        return key
    if key == "unknown":
        return key
    if key in MAKE_DISPLAY_EXCEPTIONS:
        return MAKE_DISPLAY_EXCEPTIONS[key]
    return key.title()

def canonicalize_make(series):
    keys = series.apply(make_key)
    keys = keys.replace(MAKE_ALIASES)
    keys = keys.replace(MAKE_NON_MANUFACTURER)
    return keys.apply(make_display)
```

Listing 1: Kanonikalizacija naziva proizvođača

Funkcija `make_key` obavlja prvi korak: normalizaciju zapisa. Tek nakon što su svi zapisi svedeni na isti oblik ima smisla primijeniti popise iznimaka, jer bi inače popis morao sadržavati svaku varijantu pisanja posebno.

Taj korak nije kozmetički. Bez njega bi ista marka u analizi bila razdijeljena na više vrijednosti, pa bi svaka rangirana lista proizvođača bila kriva — i to tiho, bez ijedne poruke o pogrešci.

Preimenovanje atributa. Atribut `class` preimenovan je u `vehicle_class` jer je `class`

rezervirana riječ u većini jezika i sustava koji dolaze kasnije u lancu.

Podjela skupa. Skup je podijeljen u omjeru 80:20, na 29.030 i 7.257 redaka. Podjela je slučajna, uz zadanu vrijednost generatora slučajnih brojeva, čime je ponovljiva. Njezina je svrha simulirati dva odvojena izvora podataka za potrebe procesa punjenja skladišta, opisanog u poglavlju 7: veći dio ulazi u relacijsku bazu i iz nje se kasnije čita, a manji ostaje kao zasebna datoteka.

Provjera duplikata nad cijelim skupom nije pronašla nijedan potpuno istovjetan redak, što je i očekivano jer svaki redak nosi jedinstveni identifikator detekcije.

Dva atributa namjerno ne prelaze u dimenzijski model: putanja do izvorne slike `file` i položaj kadra na vremenskoj osi videotoka `pts_ms`. Oba su tehnički trag prikupljanja, korisni za povratak do izvorne snimke, ali bez ikakve analitičke vrijednosti; u relacijskom modelu ostaju zabilježeni uz sam prolazak. Time u skladište ulazi 25 od 27 atributa, a atribut `inference_id` pritom se ne prenosi cjelovito nego rastavlja na redni broj prolaska i redni broj objekta unutar njega.

5.2 Analiza entiteta i poslovnih pravila

Skup podataka je jedna plosnata tablica: 27 stupaca u kojima su podaci o kameri, satu, prolasku, verziji modela i pojedinom vozilu izmiješani u istom retku. Zadaća relacijskog modela je prepoznati entitete koji se u toj tablici ponavljaju i razdvojiti ih tako da se svaka činjenica pohrani na jednom mjestu.

Analiza atributa otkriva šest poslovnih pravila.

Jedan prolazak može sadržavati više vozila. To je najvažnije pravilo jer određuje središnji odnos modela. Vrijednost atributa `inference_id` oblika `"1-2"` nije identifikator nego složeni ključ: prije crtice stoji redni broj prolaska, iza nje redni broj vozila unutar tog prolaska. Provjera nad cijelim skupom potvrđuje da je prefiks uvijek jednak vrijednosti stupca `motion_id`, bez ijednog odstupanja.

Svaki prolazak obradila je točno jedna verzija modela. Atribut `schema_v` veže prolazak uz verziju upute i sheme kojom je opisan, pa verzija postaje zaseban entitet, a ne stupac koji se ponavlja u svakom retku.

Kamera se nalazi na lokaciji. Podaci o uređaju i podaci o adresi opisuju dvije različite stvari: kamera se može premjestiti, a lokacija ostaje. Razdvajanje je stoga logično, iako u ovom projektu obje tablice sadrže po jedan redak.

Meteorološko opažanje vrijedi za jedan sat i jednu kameru. Ista se izmjerena vrijednost odnosi na sve prolaske unutar tog sata, pa se pohranjuje jednom, a prolasci na nju upućuju.

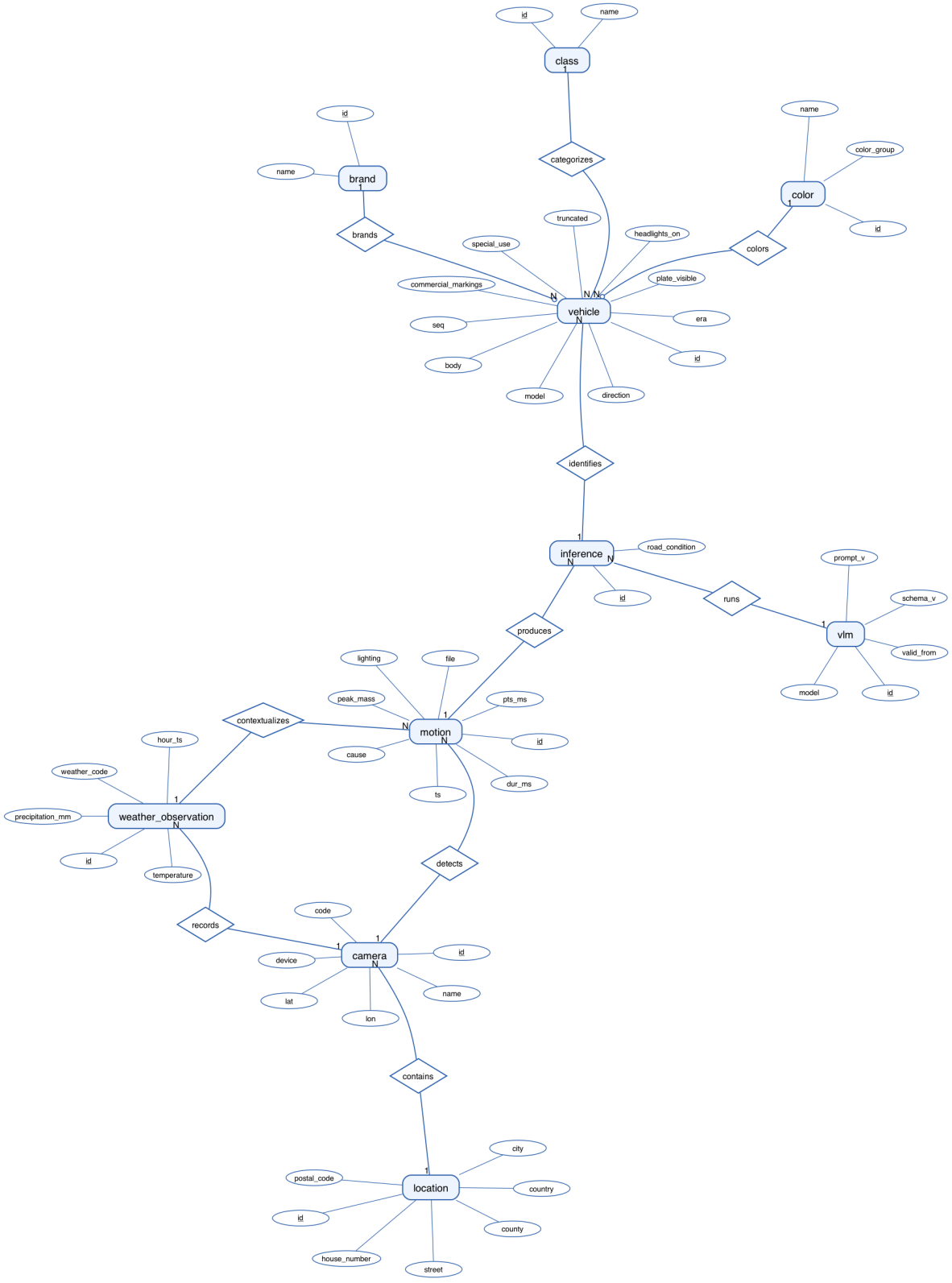
Klasa, marka i boja su šifrarnici. Riječ je o zatvorenim skupovima vrijednosti koje se ponavljaju tisućama puta. Njihovim izdvajanjem u zasebne tablice ispravak jedne

vrijednosti postaje izmjena jednog retka.

Boja pripada skupini boja. Trinaest boja svrstano je u četiri skupine — neutralne, tople, hladne i nepoznato — čime se u model unosi hijerarhija koja će u analizi omogućiti promatranje na dvije razine.

5.3 Konceptualni model

Konceptualni model prikazuje entitete i njihove odnose bez tipova podataka i ključeva. Njegova je svrha provjeriti da su odnosi ispravno postavljeni prije nego što se donese ijedna implementacijska odluka.



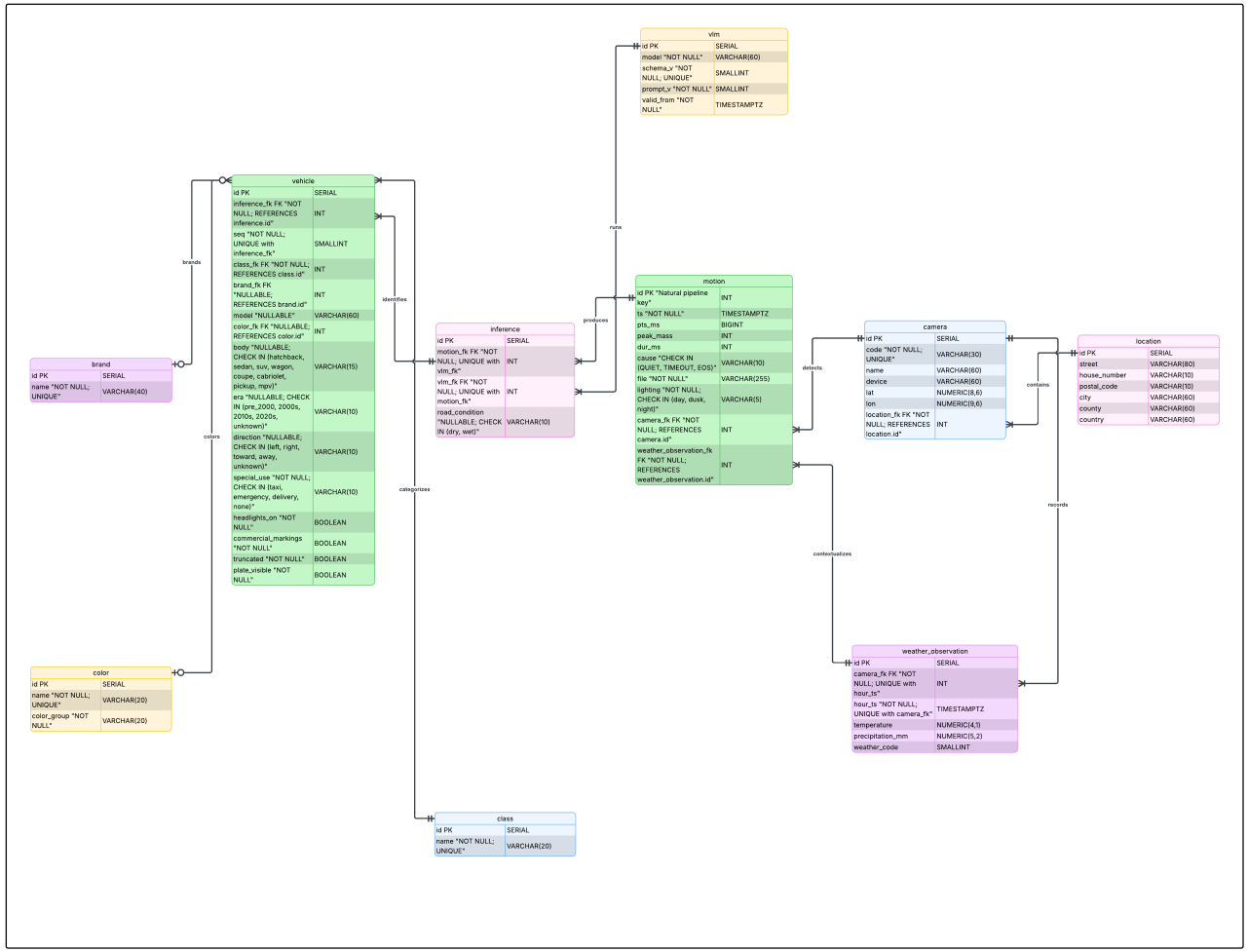
Slika 2: Konceptualni model podataka

Središnji lanac modela ide od prolaska prema vozilu: jedan prolazak ima jedno zaključivanje, a jedno zaključivanje opisuje jedno ili više vozila. Sve ostalo veže se na taj lanac — kamera i meteorološko opažanje na prolazak, verzija modela na zaključivanje, a šifrarnici

na vozilo.

5.4 Prošireni model

Prošireni model dodaje tipove podataka, primarne i strane ključeve te ograničenja. Na njemu je vidljivo i pravilo imenovanja koje se koristi kroz cijeli projekt: primarni ključevi nose naziv `id`, strani ključevi nastavak `_fk`, a kasnije će zamjenski ključevi u skladištu nositi nastavak `_tk`. Iz naziva stupca odmah se, dakle, vidi kojem sloju pripada.



Slika 3: Prošireni model podataka s tipovima i ograničenjima

5.5 Normalizacija i projektne odluke

Model je u trećoj normalnoj formi. Prelazak iz plosnate tablice u deset povezanih tablica uklonio je tri vrste ponavljanja: nazive iz šifrarnika, podatke o kameri i lokaciji te satna meteorološka mjerenja. Ilustracije radi, naziv najčešće marke koji se u ulaznoj datoteci ponavlja 1.703 puta sada postoji kao jedan jedini redak šifrarnika.

Uz normalizaciju donesene su četiri odluke koje zaslužuju obrazloženje.

5.5.1 Osvjetljenje pripada prolasku, ne satu

Atribut `lighting` na prvi pogled izgleda kao svojstvo sata i logično bi bilo smjestiti ga uz meteorološko opažanje. Mjerenje pokazuje da bi to bila pogreška: od 246 sati u skupu, njih **40 sadrži više od jedne vrijednosti osvjetljenja**. Razlog je što se osvjetljenje računa iz vremena izlaska i zalaska sunca s prijelaznim pojasom, pa unutar istog sata scena prelazi iz dana u sumrak ili iz sumraka u noć. Smještanjem atributa na sat izgubila bi se upravo ta granica — a to su sati u kojima je promet analitički najzanimljiviji.

5.5.2 Nepoznata vrijednost je redak, a ne prazno polje

Vrijednost `unknown` pohranjuje se kao redak u šifrarniku marke, boje i klase, nikada kao prazna vrijednost. Razlika je značenjska: prazno polje kaže da podatak nedostaje, a `unknown` je stvaran odgovor modela — tvrdnja da prepoznavanje nije bilo moguće. Te dvije stvari nisu isto i ne smiju se stopiti.

Praktična je posljedica ta da spajanje šifrarnika nikada ne gubi retke. Da je `unknown` bio prazna vrijednost, svako brojanje po marki tiho bi izostavilo 71 % skupa.

Prazne vrijednosti u modelu postoje, ali samo tri i svaka ima jasno značenje: `pts_ms` nedostaje u starijim zapisima, `body` se ne odnosi na vozila koja nisu automobili ili dostavna vozila, a `era` nije postojala u trećoj verziji sheme.

5.5.3 Ograničenja vrijednosti su zadržana

U model je ugrađeno sedam ograničenja tipa `CHECK` koja nabrajaju dopuštene vrijednosti za razlog zatvaranja događaja, osvjetljenje, stanje kolnika, tip karoserije, dizajnersko razdoblje, smjer i posebnu namjenu. Strogo uzevši, nisu nužna: podaci dolaze iz sheme odgovora koja je iste vrijednosti već ograničila, pa se nedopuštena vrijednost u datoteci ne bi ni trebala pojaviti.

Zadržani su svjesno, kao osigurač. Podaci u ovaj model ulaze iz datoteke koju je proizveo jezični model, a jezični modeli se mijenjaju. Ako se u nekoj budućoj verziji sheme pojavi nova vrijednost, uvoz će se zaustaviti s jasnom porukom umjesto da tiho unese vrijednost koju nijedna analiza ne očekuje. Cijena je da svako proširenje popisa vrijednosti traži

izmjenu na dva mjesta — u bazi i u modelu — i to je prihvaćena cijena.

5.5.4 Šifrarnik ne sadrži sve vrijednosti

Podjela skupa opisana u odjeljku 5.1 ima posljedicu koju treba zabilježiti. Baza se puni iz većeg dijela skupa, pa šifrarnik marki sadrži 49 od ukupno 51 vrijednosti; marke `Harley-Davidson` i `RAM` pojavljuju se isključivo u manjem dijelu.

To nije nedostatak modela. Šifrarnik obvezuje samo retke koji ulaze u relacijsku bazu, a manji dio skupa u nju ne ulazi — on se u poglavlju 7 čita izravno iz datoteke i marka se pritom preuzima kao tekst. Dimenzijski model marku i drži kao tekstualni atribut, bez zasebne tablice, pa obje vrijednosti uredno završe u skladištu.

5.6 Implementacija u sustavu PostgreSQL

Shema je opisana deklarativno, klasama u knjižnici SQLAlchemy, a stvara je skripta `03_ddl_import.py`. Pristup je odabran jer se iz istog opisa dobiva i shema baze i objektni sloj kojim se podaci upisuju, pa nema mjesta na kojem bi se ta dva razišla.

```
class Motion(Base):
    __tablename__ = "motion"
    id: Mapped[int] = mapped_column(primary_key=True, autoincrement=
       False)
    ts: Mapped[datetime] = mapped_column(DateTime(timezone=True),
       nullable=False)
    pts_ms: Mapped[Optional[int]] = mapped_column(BigInteger)
    peak_mass: Mapped[Optional[int]] = mapped_column(Integer)
    dur_ms: Mapped[Optional[int]] = mapped_column(Integer)
    cause: Mapped[Optional[str]] = mapped_column(String(10))
    file: Mapped[str] = mapped_column(String(255), nullable=False)
    lighting: Mapped[str] = mapped_column(String(5), nullable=False)
    camera_fk: Mapped[int] = mapped_column(
        ForeignKey("camera.id"), nullable=False)
    weather_observation_fk: Mapped[int] = mapped_column(
        ForeignKey("weather_observation.id"), nullable=False)
    __table_args__ = (
        CheckConstraint("cause IN ('QUIET','TIMEOUT','EOS')"),
        CheckConstraint("lighting IN ('day','dusk','night')"),
    )
```

Listing 2: Tablica prolaska s ograničenjima vrijednosti

Vrijedi obratiti pozornost na dvije stvari. Primarni ključ je označen s `autoincrement=False` jer redni broj prolaska nije nova vrijednost koju baza dodjeljuje, nego onaj isti broj koji je dodijelio sustav za prikupljanje — veza prema izvornoj slici time ostaje sačuvana. I ograničenja su dio definicije tablice, a ne naknadni dodatak, pa se stvaraju istim pozivom kao i sama tablica.

Uvoz se odvija u dva koraka. Najprije se popunjavaju šifrarnici i tablice s malim brojem redaka, a zatim se u jednom prolazu kroz skup upisuju prolasci, zaključivanja i vozila. Redoslijed je nužan jer strani ključevi moraju pokazivati na retke koji već postoje. Skripta prije stvaranja sheme briše postojeću, čime je cijeli uvoz ponovljiv: isti ulaz uvijek daje istu bazu.

5.7 Uvoz podataka i validacija

Nakon uvoza baza sadrži deset tablica prikazanih u tablici 9.

Tablica 9: Broj redaka po tablici relacijskog modela

Tablica	Redaka	Sadržaj
`location`	1	adresa mjerne lokacije
`camera`	1	uređaj i koordinate
`vlm`	2	verzije modela i sheme
`class`	8	šifrarnik klasa vozila
`color`	13	šifrarnik boja sa skupinama
`brand`	49	šifrarnik marki
`weather_observation`	246	satna meteorološka mjerenja
`motion`	28.435	prolasci
`inference`	28.435	zaključivanja
`vehicle`	29.030	pojedinačna vozila

Odnos zadnja tri broja potvrđuje središnje poslovno pravilo. Prolazaka i zaključivanja ima jednako mnogo jer je svaki prolazak obrađen točno jednom, dok vozila ima 595 više — toliko je prolazaka sadržavalo više od jednog vozila.

Provjera uvoza provedena je u dva stupnja. Prvi je ručni: upit u skripti `05_verify_import.sql` spaja svih deset tablica i rekonstruira izvorne retke, tako da se prvih stotinu može usporediti s datotekom očima.

Drugi je stupanj automatski i on je stvarni dokaz. Skripta `04_import_check.py` istim upitom rekonstruira *sve* retke iz baze i provodi dvije provjere: podudaranje popisa stupaca i cjelovitu usporedbu tablica.

```
class TestDatabase(unittest.TestCase):
    def setUp(self):
        self.engine = sqlalchemy.create_engine(DATABASE_URL)
        self.connection = self.engine.connect()

        self.df = pd.read_csv(CSV_FILE_PATH)

        result = self.connection.execute(text(QUERY))
        self.db_df = pd.DataFrame(result.fetchall())
```

```
        self.db_df.columns = list(result.keys())

    def test_columns(self):
        self.assertListEqual(list(self.df.columns),
                             list(self.db_df.columns))

    def test_dataframes(self):
        df = normalize(self.df).reset_index(drop=True)
        db_df = normalize(self.db_df).reset_index(drop=True)
        assert_frame_equal(df, db_df)
```

Listing 3: Provjera da normalizacija nije izgubila podatke

Prije usporedbe funkcija `normalize` usklađuje tipove podataka na obje strane — vremenska oznaka svodi se na istu vremensku zonu, brojčani stupci na isti brojčani tip, tekstualni na isti tekstualni. To ne ublažava provjeru: uspoređuju se vrijednosti, a usklađivanje tipova samo sprječava da usporedba padne zato što čitanje datoteke i čitanje iz baze isti broj vraćaju u različitom zapisu.

Usporedba prolazi nad svih 29.030 redaka i svih 27 stupaca. Riječ je o redcima većeg dijela skupa — baza je iz njega i napunjena — pa je usporedba potpuna: provjereno je sve što je u bazu ušlo.

To je konkretan dokaz da je normalizacija bila bez gubitka. Iz deset tablica moguće je spajanjem vratiti točno onaj plosnati oblik od kojeg se krenulo, redak po redak i vrijednost po vrijednost. Nijedan podatak nije nestao, nijedan se nije promijenio, a ponavljanja više nema.

6 Dimenzijski model podataka

Relacijski model iz prethodnog poglavlja vjerno opisuje stvarnost i ne sadrži nijedno suvišno ponavljanje, ali za analitičke upite nije prikladan. Odgovor na pitanje koliko je dvokotača prošlo po satu za kišnih dana zahtijeva spajanje šest tablica, a takvih pitanja u analizi ima na desetke. Dimenzijski model isti sadržaj organizira drukčije: mjere se skupljaju u središnju tablicu, a sve što ih opisuje razvrstava se u dimenzije koje su namjerno denormalizirane. Podaci su isti, način pohrane nije.

Poglavlje odgovara trećoj kontrolnoj točki, a model stvara skripta `generate_star_schema.py` u zasebnoj shemi `dw` iste baze.

6.1 Postupak dizajna

Model je izgrađen u četiri koraka: odabir poslovnog procesa, određivanje zrnatosti tablice činjenica, definiranje dimenzija i naposljetku definiranje mjera. Redoslijed nije proizvoljan

— zrnatost se određuje prije dimenzija jer upravo ona određuje što pojedina dimenzija smije opisivati.

Poslovni proces koji se modelira jest *prolazak vozila kroz mjernu zonu*. Sve u modelu odgovara na jedno pitanje: koliko je vozila prošlo, kada, kakvih i u kojim uvjetima.

6.2 Zrnatost tablice činjenica

Zrnatost je određena na najnižoj mogućoj razini: **jedan redak tablice činjenica jedno je detektirano vozilo**. To nije isto što i jedan prolazak. Prolazak je događaj koji je zabilježila kamera, a unutar jednog prolaska model može prepoznati više vozila — automobil i motocikl koji istodobno uđu u zonu dijele isti redni broj prolaska, ali daju dva retka.

Brojke to potvrđuju: tablica činjenica sadrži 36.287 redaka nastalih iz 35.360 prolazaka. Od toga 902 prolaska sadrže više od jednog vozila, a najveći ih sadrži pet.

Odluka da se ide na najnižu razinu ima izravnu posljedicu. Da je zrnatost postavljena na prolazak, svaki bi upit o klasama vozila bio kriv, jer bi prolazak s automobilom i motociklom morao dobiti samo jednu klasu. Na razini detekcije nijedno vozilo se ne gubi i nijedno se ne broji dvaput.

6.3 Tablica činjenica

Tablica `fact_vehicle_detection` pripada **transakcijskom modelu** činjeničnih tablica. Od tri moguća modela — transakcijskog, periodičke snimke stanja i akumulirajuće snimke stanja — transakcijski je onaj koji bilježi pojedinačne događaje u trenutku njihova nastanka i ne mijenja ih poslije.

To odgovara prirodi podataka. Prolazak vozila je trenutačan događaj: dogodi se, zabilježi se i više se ne mijenja. Ne postoji stanje koje bi se povremeno snimalo, kao ni proces koji bi kroz faze napredovao pa se isti redak nadopunjavao. Redak nastaje jednom i ostaje takav.

6.3.1 Mjere

Tablica nije bez činjenica — sadrži tri mjere:

- `peak_mass` — vršno zauzeće zone u pikselima, posredni pokazatelj veličine vozila;
- `dur_ms` — trajanje zadržavanja u zoni u milisekundama, posredni pokazatelj brzine;
- `objects_in_event` — broj vozila prepoznatih u istom prolasku.

Prve dvije mjere su aditivne: zbroj i prosjek imaju smisla po bilo kojoj kombinaciji dimenzija. Treća nije. Vrijednost `objects_in_event` jednaka je za sve retke istog prolaska, pa bi njezino zbrajanje isti prolazak brojalo onoliko puta koliko u njemu ima vozila. Ona služi kao pokazatelj gustoće prometa u trenutku detekcije, a ne kao veličina koju se zbraja.

6.3.2 Degenerirana dimenzija

Atribut `motion_id` nalazi se u tablici činjenica, ali nije ni mjera ni surogat ključ koji pokazuje na neku dimenzijsku tablicu. Riječ je o *produkcijskom* ili prirodnom ključu — rednom broju prolaska koji je dodijelio sustav za prikupljanje i koji je u model prenesen nepromijenjen. Takav atribut ima sva svojstva dimenzijskog ključa, ali odgovarajuća dimenzijska tablica ne postoji, pa se naziva **degeneriranom dimenzijom**.

Razlog je što bi ta tablica bila prazna. Sve što opisuje prolazak već je razvrstano drugdje — vrijeme u dimenziju datuma i vremena, uvjeti u dimenziju uvjeta, kamera u dimenziju kamere, način zatvaranja događaja u dimenziju promatranja. Kad se sve to oduzme, dimenziji prolaska ostao bi samo vlastiti ključ i ništa uz njega. Takva tablica ne bi donijela nijedan atribut, a svaki bi upit koštala jednim spajanjem više.

Zadržan je iz dva razloga. Prvi je grupiranje: njime se retci istog prolaska prepoznaju kao cjelina, što je bilo nužno već pri utvrđivanju zrnatosti. Drugi je veza prema izvoru — dimenzijski model surogat ključevima namjerno prekida vezu s produkcijskim ključevima, pa je `motion_id` jedini put natrag do izvorne slike i njezina zapisa u sustavu za prikupljanje.

6.4 Dimenzije i surogat ključevi

Model sadrži sedam dimenzija. Svaka ima vlastiti surogat ključ s nastavkom `_tk` — cjelobrojnu sekvencu koja nema nikakvo značenje osim identifikacije i nije povezana s produkcijskim ključem izvora.

Tablica 10: Dimenzije zvjezdaste sheme

Dimenzija	Redaka	Sadržaj
`dim_vehicle`	3.015	klasa, marka, model, boja, skupina boje, karoserija, razdoblje
`dim_time`	1.413	sat, minuta, doba dana
`dim_observation`	103	okolnosti pojedinog opažanja
`dim_conditions`	31	vremenske prilike, temperatura, oborine, osvjetljenje
`dim_date`	12	datum, dan, dan u tjednu, vikend, mjesec, kvartal, godina
`dim_vlm`	2	verzija modela i upute
`dim_camera`	1	uređaj, koordinate i adresa

Razlika u veličini je očekivana i poželjna. Dimenzija vozila ima tri tisuće redaka jer opisuje mnogo kombinacija svojstava, a dimenzija kamere jedan redak jer je sustav jednokameran. Obje su jednako valjane dimenzije.

6.5 Hijerarhije

U model je ugrađeno pet hijerarhija, raspoređenih u četiri dimenzije. Dimenzija vozila nosi dvije, što je primjer višestrukih hijerarhija unutar iste dimenzijske tablice.

Tablica 11: Hijerarhije u dimenzijskim tablicama

Dimenzija	Hijerarhija	Razine
`dim_vehicle`	vrsta vozila	klasa → marka → model
`dim_vehicle`	boja	skupina boje → boja
`dim_date`	kalendar	godina → kvartal → mjesec → dan
`dim_time`	vrijeme	doba dana → sat → minuta
`dim_camera`	geografija	država → županija → grad → ulica

Hijerarhije nisu ukras. One su ono što u analizi omogućuje spuštanje i podizanje razine promatranja, o čemu se govori u poglavlju 8. Bez njih bi svaka promjena razine detalja bila novi upit umjesto jednog klika.

6.6 Napredne izvedbe dimenzijskih tablica

Četiri dimenzije u modelu nisu obične dimenzijske tablice nego primjeri naprednih izvedbi.

6.6.1 Sporo mijenjajuća dimenzija tipa 2

Dimenzija `dim_vlm` rješava pitanje koje stoji u srcu svake sporo mijenjajuće dimenzije: *što učiniti s atributom kada mu se vrijednost promijeni?*

Promjena se doista dogodila. Tijekom prikupljanja uputa i shema odgovora prešle su iz treće u četvrtu verziju, čime je uveden atribut dizajnerskog razdoblja i pooštreno pravilo razvrstavanja dostavnih vozila. Nakon 27. kolovoza 2026. u 11:46 detekcije su nastajale drukčijim postupkom nego prije toga.

Od četiri moguća pristupa odabran je **tip 2**: promjena se bilježi kao novi redak dimenzije, uz atribute koji određuju razdoblje njegove valjanosti. Stari redak dobiva datum do kojeg je vrijedio, novi ostaje otvoren.

Tablica 12: Sadržaj dimenzije `dim_vlm`

vlm_tk	verzija	vrijedi od	vrijedi do	shema
1	1	20. 08. 2026. 21:26	27. 08. 2026. 11:46	3
2	2	27. 08. 2026. 11:46	—	4

Ostala tri pristupa ovdje ne bi bila dobra. Tip 1 presnimava staru vrijednost novom i time bi obrisao podatak o tome da je trećina skupa nastala starijom verzijom. Tip 3 pamti samo prethodnu vrijednost u dodatnom stupcu i ne bi podnio treću verziju. Tip 4 povijest seli u zasebnu tablicu, što bi za dva retka bilo neproporcionalno složeno.

Tip 2 daje točno ono što treba: strani ključ u tablici činjenica pokazuje na verziju koja je proizvela taj redak, pa se u analizi u svakom trenutku vidi kojoj verziji koja detekcija pripada. Bez toga usporedba kvalitete dviju verzija iz poglavlja 8 ne bi bila moguća.

6.6.2 Kompozitna dimenzija

Dimenzija `dim_observation` kompozitna je dimenzija — u literaturi poznata i kao *junk dimension*. Nastala je premještanjem atributa niske kardinalnosti iz tablice činjenica u zasebnu dimenziju.

Riječ je o osam atributa: četiri logičke oznake (odsječenost, vidljivost registarske pločice, upaljena svjetla, poslovne oznake) i četiri kratka pokazatelja (razlog zatvaranja događaja, smjer, posebna namjena, stanje kolnika). Nijedan od njih ne bi opravdao vlastitu dimenziju, a svi zajedno u tablici činjenica zauzimali bi osam stupaca u svakom od 36.287 redaka.

Kombinacija tih osam atributa teorijski daje preko tisuću mogućih vrijednosti. U stvarnim podacima pojavljuje se **samo 103 kombinacije**, i upravo toliko redaka dimenzija i sadrži. Tablica činjenica umjesto osam stupaca nosi jedan strani ključ.

Za razliku od mini-dimenzije, atributi ove dimenzije nisu međusobno logički povezani. Njihova je zajednička osobina samo to što su svi kratki i svi opisuju okolnosti pojedinog opažanja.

6.6.3 Heterogena dimenzija

Dimenzija `dim_vehicle` ima svojstva heterogene dimenzije: u jednu tablicu spaja različite, ali logički srodne entitete. Automobil, bicikl i turistički vlak sve su to vozila, ali ih ne opisuju isti atributi.

To se najjasnije vidi na tipu karoserije. Mjerenje nad dimenzijom pokazuje da je taj atribut popunjen za 2.344 od 2.361 retka klase `car` i za 528 od 541 retka klase `van`, a za svih ostalih šest klasa popunjen je u **točno nula redaka**. Bicikl nema tip karoserije, i to nije nedostatak podatka nego svojstvo entiteta.

Alternativa bi bila razdvajanje u zasebne dimenzije po klasi vozila, čime bi svaka analiza koja obuhvaća sav promet zahtijevala uniju više tablica. Odabrana je jedna dimenzija s opcionalnim atributima — ravnoteža između jedne velike polupopunjene dimenzije i mnogo malih.

6.6.4 Grupiranje kvantitativnih vrijednosti

Dimenzija `dim_conditions` nastala je postupkom koji dimenzijsko modeliranje izričito predviđa: pretvorbom kvantitativnih vrijednosti u kvalitativne grupiranjem.

Izvorni podaci sadrže temperaturu u stupnjevima, količinu oborina u milimetrima i šifru vremena. Sve tri su brojčane veličine, ali nijedna nije mjera — one ne opisuju prolazak vozila nego okolnosti u kojima se dogodio, pa im mjesto nije u tablici činjenica. Svaka je zato svedena na kvalitativnu vrijednost, i to zasebnim pravilom:

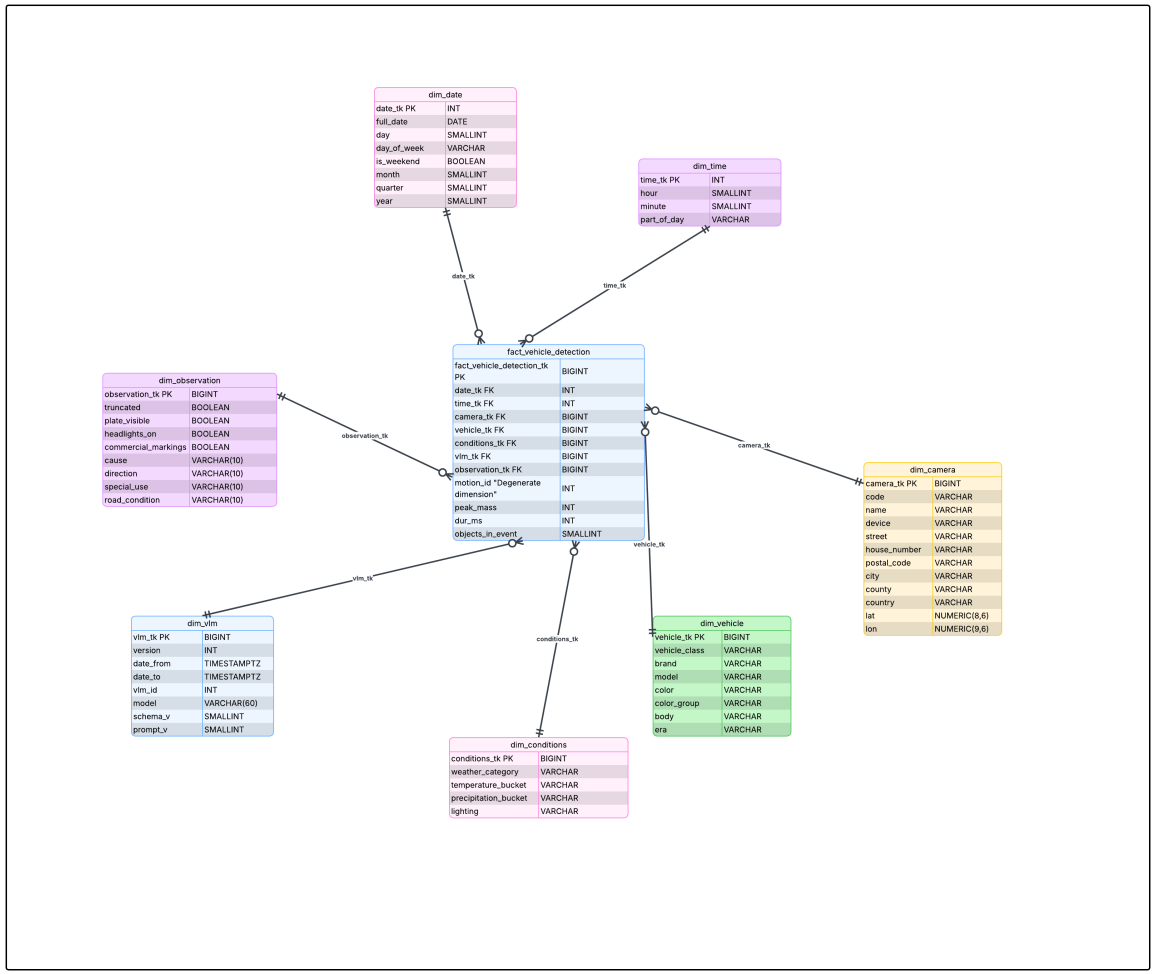
- temperatura se dijeli u četiri razreda po visini, s granicama na 20, 25 i 30 stupnjeva;
- količina oborina u četiri razreda po jačini, s granicama na 0, 0,5 i 2,5 milimetra;
- šifra vremena preslikava se rječnikom izvedenim iz standarda Svjetske meteorološke organizacije, kojim se izvorne šifre svode na kategorije poput vedro, oblačno, rosulja i kiša.

Četvrti atribut dimenzije, osvjetljenje, ne prolazi kroz to grupiranje jer je već kvalitativan — u dimenziju ulazi nepromijenjen, onakav kakav je izračunat iz vremena izlaska i zalaska sunca u odjeljku 3.8.

Rezultat je dimenzija od 31 retka koja pokriva sve kombinacije uvjeta zabilježene tijekom prikupljanja.

6.7 Zvjezdasta shema

Tablica činjenica u središtu, sedam dimenzija oko nje, veze isključivo tipa 1:M od dimenzije prema činjenicama — to je zvjezdasta shema. Nijedna dimenzija ne referencira drugu dimenziju, pa nema pahuljičastih ogranaka.



Slika 4: Zvjezdasta shema skladišta podataka

Shema je opisana istim deklarativnim pristupom kao i relacijski model.

```
class FactVehicleDetection(Base):
    __tablename__ = "fact_vehicle_detection"
    __table_args__ = {"schema": SCHEMA}

    fact_vehicle_detection_tk: Mapped[int] = mapped_column(
        BigInteger, primary_key=True)

    date_tk: Mapped[Optional[int]] = mapped_column(
        Integer, ForeignKey(f"{SCHEMA}.dim_date.date_tk"))
    time_tk: Mapped[Optional[int]] = mapped_column(
        Integer, ForeignKey(f"{SCHEMA}.dim_time.time_tk"))
    camera_tk: Mapped[Optional[int]] = mapped_column(
        BigInteger, ForeignKey(f"{SCHEMA}.dim_camera.camera_tk"))
    vehicle_tk: Mapped[Optional[int]] = mapped_column(
        BigInteger, ForeignKey(f"{SCHEMA}.dim_vehicle.vehicle_tk"))
    conditions_tk: Mapped[Optional[int]] = mapped_column(
        BigInteger, ForeignKey(f"{SCHEMA}.dim_conditions.conditions_tk
            "))
    vlm_tk: Mapped[Optional[int]] = mapped_column(
        BigInteger, ForeignKey(f"{SCHEMA}.dim_vlm.vlm_tk"))
    observation_tk: Mapped[Optional[int]] = mapped_column(
        BigInteger, ForeignKey(f"{SCHEMA}.dim_observation.
            observation_tk"))

    motion_id: Mapped[Optional[int]] = mapped_column(Integer)
    peak_mass: Mapped[Optional[int]] = mapped_column(Integer)
    dur_ms: Mapped[Optional[int]] = mapped_column(Integer)
    objects_in_event: Mapped[Optional[int]] = mapped_column(
       SmallInteger)
```

Listing 4: Tablica činjenica sa surogat ključevima, degeneriranom dimenzijom i mjerama

Struktura tablice čita se izravno iz definicije: vlastiti surogat ključ, sedam stranih ključeva — po jedan prema svakoj dimenziji — zatim degenerirana dimenzija i tri mjere. Ništa drugo u njoj ne postoji.

6.8 Prednosti zvjezdastog modela

Usporedba s relacijskim modelom pokazuje što se dobilo. Pitanje o broju dvokotača po satu za kišnih dana u relacijskom modelu traži spajanje šest tablica; u zvjezdastoj shemi traži spajanje tablice činjenica s tri dimenzije, a svi potrebni atributi već su u njima.

Druga je prednost razumljivost. Dimenzije su korisničko sučelje modela: naziv `dim_vehicle` i njegovi atributi razumljivi su bez poznavanja načina na koji su podaci prikupljeni. Upravo to omogućuje da se u alatu za vizualizaciju model koristi izravno, bez pisanja ijednog upita.

Cijena je redundancija. Naziv marke koji je u relacijskom modelu bio jedan redak šifrarnika sada se ponavlja u dimenziji vozila onoliko puta koliko postoji kombinacija s tom markom. To je svjesno prihvaćeno: prostor je jeftin, a spajanja u analitičkim upitima nisu.

7 ETL proces

Prethodna su dva poglavlja stvorila dva modela istih podataka: relacijski, prilagođen zapisivanju pojedinačnih događaja, i dimenzijski, prilagođen njihovoj analizi. Dimenzijski model je do sada bio prazna konstrukcija — shema `dw` sadržavala je definicije tablica, ali nijedan redak. Ovo poglavlje opisuje postupak kojim se ta konstrukcija puni podacima. Poglavlje odgovara četvrtoj kontrolnoj točki.

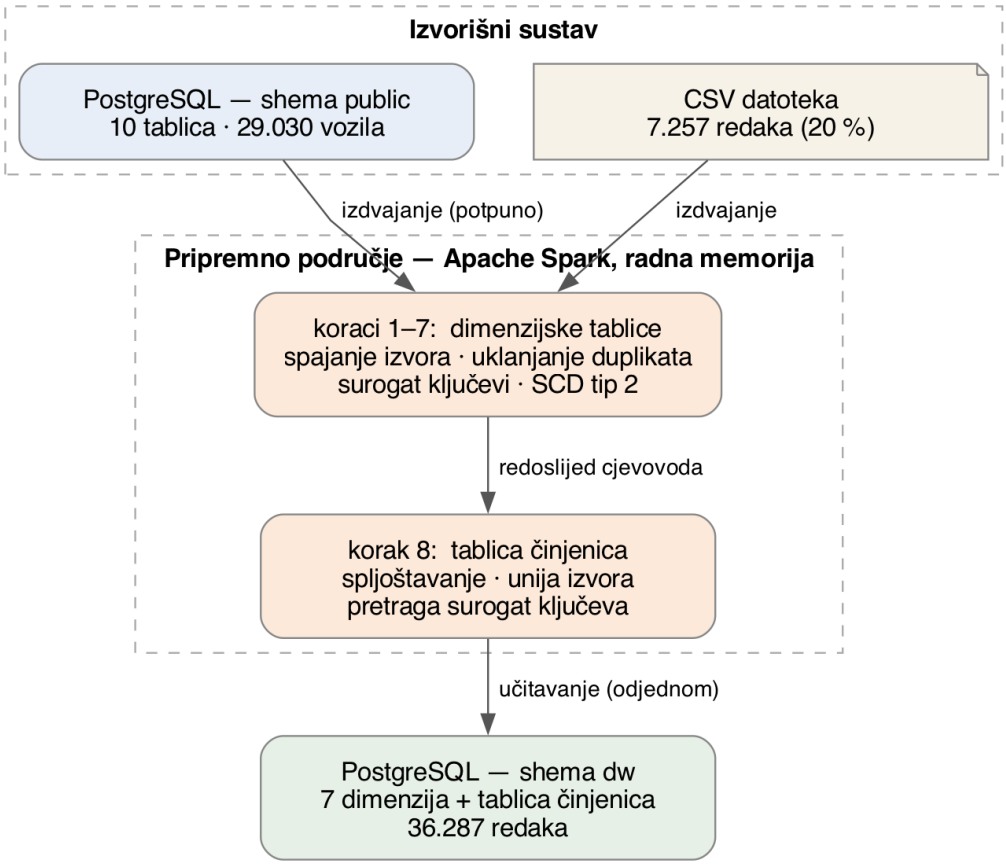
7.1 Arhitektura ETL rješenja

Postupak punjenja dimenzijskog modela naziva se ETL proces, prema trima osnovnim operacijama od kojih se sastoji: izdvajanju podataka iz izvorišnog sustava (*extract*), njihovoj transformaciji (*transform*) i učitavanju u skladište podataka (*load*) [4]. Potreba za takvim procesom proizlazi iz same razlike između dvaju modela. Podaci u izvorišnom sustavu zapisani su u obliku koji odgovara njihovu nastanku: normalizirani, razdvojeni u šifrarnike, bez izvedenih vrijednosti. Dimenzijski ih model traži u bitno drugačijem obliku: spljoštene, denormalizirane, s izvedenim atributima i surogat ključevima. Nijedan od tih oblika nije pogrešan — svaki je ispravan za svoju namjenu — ali prijelaz iz jednog u drugi netrivijalan je posao koji ne obavlja ni izvorišni sustav ni skladište, nego proces između njih.

Kao alat odabran je Apache Spark, odnosno njegovo sučelje za Python [5]. Odabir slijedi opću promjenu u praksi izrade skladišta podataka, u kojoj se grafički alati za ETL postupno napuštaju u korist rješenja opisanih programskim kodom. Prednost je programskog pristupa u tome što je ETL proces time obična programska datoteka: može se čitati, uspoređivati po verzijama i ponovno pokrenuti bez posredovanja grafičkog sučelja. Spark je uz to izvorno namijenjen radu s količinama podataka koje ne stanu u memoriju jednog računala, pa isti kod koji ovdje obrađuje desetke tisuća redaka ne treba mijenjati ako bi ih bilo nekoliko redova veličine više.

Okruženje je postavljeno na jednom mjestu. Spark se pokreće kao jedinstvena radna sesija (`SparkSession`) koja se dijeli među svim koracima procesa, pri čemu se uz sesiju vežu tri postavke: putanja do JDBC upravljačkog programa za PostgreSQL, vremenska zona `Europe/Zagreb` te varijabla `JAVA_HOME` usmjerena na Javu inačice 17. Postavljanje vremenske zone nije formalnost: iz vremenske oznake prolaska izvode se kalendarski datum i sat, a bez izričito postavljene zone te bi se vrijednosti računale prema zadanoj zoni izvršnog okruženja, a ne prema lokalnom vremenu mjerne lokacije. Vezanje uz Javu

17 posljedica je toga što upotrijebljena inačica Sparka (3.5) podržava Javu zaključno s inačicom 17, dok je na razvojnom računalu zadana bila novija.



Slika 5: Tijek ETL procesa: dva izvora, pripremno područje i skladište podataka

Središnji pojam u toj arhitekturi jest **pripremno područje** (engl. *staging area*). Riječ je o prostoru u koji se izdvojeni podaci odlažu prije transformacije i iz kojega se, nakon nje, puni skladište. U ovom rješenju pripremno područje nije zasebna baza ni shema na disku, nego skup podatkovnih okvira (*DataFrame*) u radnoj memoriji Sparkove sesije. Takva izvedba zadovoljava sva svojstva koja se od pripremnog područja traže: ono je kopija izvorišnih podataka, korisnici mu nemaju pristup, izvana se nad njim ne mogu izvršavati upiti, a okviri se u njemu stvaraju i odbacuju bez ikakvih pravila. Dodatno svojstvo koje proizlazi iz odabira Sparka jest odgođeno izvršavanje: sve dok se ne zatraži zapisivanje, okviri postoje samo kao plan izvršavanja, a ne kao materijalizirani podaci. Stvarni prolazak kroz podatke događa se tek u fazi učitavanja.

Sam proces vodi jedna upravljačka datoteka koja poziva tri faze redom — izdvajanje, transformaciju i učitavanje — i pritom na početku prazni predmemoriju sesije kako se ne bi upotrijebio međurezultat prethodnog pokretanja.

7.2 Faza izvlačenja (extract)

Zahtjev četvrte kontrolne točke jest da se skladište napuni iz najmanje dva izvora. Taj je zahtjev ovdje ispunjen na način koji nije umjetan, nego proizlazi iz odluke donesene još u poglavlju 5: skup podataka podijeljen je slučajnim odabirom u omjeru 80:20, pri čemu je veći dio uvezen u relacijsku bazu, a manji ostavljen kao datoteka. ETL proces zato izdvaja podatke iz dvaju stvarno različitih izvora:

- **Relacijska baza podataka** — shema `public` sustava PostgreSQL, iz koje se preko JDBC sučelja dohvaća svih deset tablica relacijskog modela, ukupno 29.030 zapisa o vozilima s pripadajućim prolascima, meteorološkim opažanjima i šifrarnicima.
- **Datoteka s razdvojenim vrijednostima** — preostalih 7.257 redaka u obliku CSV datoteke, koja se čita izravno, sa zaglavljem i automatskim zaključivanjem tipova stupaca.

Dva se izvora razlikuju u više od formata. Baza podataka podatke isporučuje normalizirano, razdvojene u deset tablica koje treba spojiti; datoteka ih isporučuje spljoštene, u 27 stupaca. Za dio atributa razlikuju se i nazivi — ono što je u datoteci `make`, u bazi je `brand.name`, a `temp_c` odgovara stupcu `weather_observation.temperature`. Usklađivanje tih razlika prvi je posao faze transformacije i razlog zbog kojeg se svaka transformacija piše u dvije varijante, po jednoj za svaki izvor.

```
from spark_session import get_spark_session

JDBC_URL = "jdbc:postgresql://localhost:5432/srp-traffic-analysis"
CONNECTION_PROPERTIES = {
    "user": "adisspahic",
    "driver": "org.postgresql.Driver",
}


def extract_table(table_name):
    spark = get_spark_session("ETL_App")

    df = spark.read.jdbc(
        url=JDBC_URL,
        table=f"public.{table_name}",
        properties=CONNECTION_PROPERTIES,
    )
    return df


def extract_all_tables():
    return {
        "location": extract_table("location"),
        "camera": extract_table("camera"),
        "vlm": extract_table("vlm"),
```

```
        "weather_observation": extract_table("weather_observation"),
        "motion": extract_table("motion"),
        "inference": extract_table("inference"),
        "class": extract_table("class"),
        "brand": extract_table("brand"),
        "color": extract_table("color"),
        "vehicle": extract_table("vehicle"),
    }
```

Listing 5: Izdvajanje iz relacijske baze preko JDBC sučelja

Prema načinu izdvajanja razlikuju se potpuno i djelomično izdvajanje. Ovdje je provedeno **potpuno izdvajanje**: iz svake se tablice dohvaćaju svi dostupni podaci, bez ikakvog uvjeta koji bi ograničio dohvat na promijenjene retke. To je ujedno i *prvo punjenje* skladišta podataka — skladište prije ovog procesa nije sadržavalo nijedan redak, pa se pitanje odnosa novih i već učitanih podataka ne postavlja. Prvo je punjenje redovito jednostavnije od kasnijih osvježavanja upravo zato što se ne mora voditi računa o postojećem sadržaju.

Djelomično izdvajanje, koje bi dolazilo u obzir pri periodičkom osvježavanju, tražilo bi mehanizam za otkrivanje promjena u izvorišnom sustavu (*Change Data Capture*). U ovom projektu takav mehanizam nije potreban, ali vrijedi primijetiti da bi izvorišni sustav bio pogodan za najjednostavniju od tih tehnika — dodavanje vremenske oznake. Tablica `motion` već nosi stupac `ts`, zapisi se u nju isključivo dodaju, nikada ne mijenjaju niti brišu, pa bi usporedba te oznake s vremenom posljednjeg uspješnog izdvajanja bila dovoljna. Uobičajena ograničenja te tehnike — da bilježi samo posljednju izmjenu i da ne prepoznaje obrisane zapise — ovdje ne bi predstavljala problem jer izmjena i brisanja u izvoru nema.

Poseban dio faze izdvajanja jest briga o izvorišnom sustavu. Načelo je da produkcijski sustav ne smije biti ugrožen prikupljanjem podataka za skladište, iz čega slijedi nekoliko praktičnih pravila. Prvo, ETL proces ne smije mijenjati ni brisati podatke u izvoru; ovdje je to zadovoljeno time što se izvorišnoj bazi pristupa isključivo za čitanje, a jedino se zapisivanje u cijelom procesu događa u odredišnu shemu. Drugo, konfiguracija izvorišnog sustava ne bi se trebala mijenjati, a ako je izmjena nužna, treba je svesti na najmanju moguću mjeru; izdvajanje se ovdje oslanja samo na standardno JDBC sučelje i postojećeg korisnika baze, bez ijedne izmjene u izvoru. Treće, potrebno je voditi računa o performansama izvorišnog sustava, jer potpuno izdvajanje čita cijele tablice i time ih zauzima; u produkcijskom bi se sustavu takav dohvat rasporedio u vrijeme kada je opterećenje najmanje, u ovom konkretnom slučaju noću, kada je promet na mjernoj lokaciji najrjeđi, a broj korisnika sustava najmanji.

Valja biti precizan oko toga koliko ta razmatranja vrijede za ovaj projekt. Izvorišna baza ovdje nije živi produkcijski sustav s više istodobnih korisnika, nego baza stvorena u drugoj kontrolnoj točki, koja preuzima ulogu izvorišnog sustava. Navedena su pravila stoga u ovom procesu poštovana, ali nisu bila i podvrgnuta stvarnom ispitu opterećenja.

7.3 Faza transformacije (transform)

Transformacija se u cijelosti odvija u pripremnom području i obuhvaća dvije skupine poslova: čišćenje podataka i njihovo oblikovanje u strukturu koju traži dimenzijski model. Prva je skupina u ovom projektu razmjerno lagana, i to zato što je najveći dio čišćenja obavljen ranije — ručnom revizijom opisanom u odjeljku 3.9 i pripremom podataka opisanom u odjeljku 5.1. Podaci koji ulaze u ETL proces već su prošli uklanjanje duplikata kadra, ispravak klasa, kanonikalizaciju naziva proizvođača i zamjenu nepouzdanih atributa vanjskim izvorom. Ono što u ovoj fazi preostaje jest usklađivanje dvaju izvora i izvođenje vrijednosti kojih u izvoru nema.

Druga je skupina poslova opsežnija i obuhvaća četiri postupka koje dimenzijski model traži: uklanjanje duplih dimenzijskih zapisa, mehanizam sporo mijenjajućih dimenzija, generiranje surogat ključeva i izgradnju tablice činjenica s vezama prema dimenzijama.

Operacije se izvršavaju slijednim redoslijedom (*pipelining order*), u osam koraka. Redoslijed nije proizvoljan: prvih sedam koraka gradi dimenzijske tablice, a osmi gradi tablicu činjenica. Tablica činjenica mora biti posljednja jer njezini stupci sadrže surogat ključeve dimenzija, a ti ključevi nastaju tek u koracima koji joj prethode.

Tablica 13: Koraci cjevovoda transformacije i izvori iz kojih se pune

Korak	Tablica	Izvori	Postupak
1	`dim_date`	oba	izvođenje kalendarskih atributa iz datuma
2	`dim_time`	oba	izvođenje sata, minute i dijela dana
3	`dim_camera`	baza	spajanje kamere s lokacijom
4	`dim_vehicle`	oba	spajanje šifrarnika, denormalizacija
5	`dim_conditions`	oba	grupiranje kvantitativnih vrijednosti
6	`dim_observation`	oba	kombiniranje osam obilježja
7	`dim_vlm`	baza	sporo mijenjajuća dimenzija tipa 2
8	`fact_vehicle_detection`	oba	spljoštavanje i pretraga ključeva

7.3.1 Transformacija dimenzijskih tablica

Svaka dimenzijska transformacija slijedi isti obrazac u tri koraka: iz svakog se izvora izgradi okvir s istim skupom stupaca, ta se dva okvira spoje u jedan, a zatim se iz spojenog okvira ukloni sve što se ponavlja i dodijeli surogat ključ.

Uklanjanje duplih dimenzijskih zapisa temeljni je korak izgradnje dimenzije i ujedno onaj koji objašnjava zašto su dimenzijske tablice mnogo manje od tablice činjenica. Dimenzija ne opisuje događaje nego njihova svojstva, pa je za nju bitna svaka *kombinacija* vrijednosti, a ne svako pojavljivanje te kombinacije. Iz 36.287 zapisa o vozilima ostaje tako 3.015 različitih kombinacija klase, marke, modela, boje, skupine boje, karoserije i razdoblja. Kombinacija koja se u prometu javlja tisuću puta u dimenziji zauzima jedan redak, na koji tisuću redaka tablice činjenica pokazuje istim ključem.

Uklanjanje duplikata mora se pritom provesti *nakon* spajanja izvora, a ne prije, i razlog je u tome što se skupovi kombinacija dvaju izvora uvelike preklapaju. Iz baze podataka proizlazi 2.711 različitih kombinacija vozila, iz datoteke njih 1.320, ali 1.016 kombinacija javlja se u oba izvora, pa je različitih ukupno 3.015. Kada bi se duplikati uklanjali zasebno po izvoru, tih bi 1.016 kombinacija ostalo u dimenziji dvaput, svaki put s vlastitim surogat ključem, i dimenzija bi imala 4.031 redak umjesto 3.015. Posljedica ne bi bio tek suvišan redak. Pri izgradnji tablice činjenica svaka detekcija s takvom kombinacijom pronašla bi *dva* odgovarajuća retka dimenzije i time bi se udvostručila; kako se 32.967 od 36.287 detekcija, odnosno 90,9 %, odnosi upravo na kombinacije prisutne u oba izvora, tablica činjenica narasla bi na 69.254 retka — gotovo dvostruko više nego što je vozila stvarno prošlo. Upravo takvo umnožavanje otkriva druga provjera opisana u odjeljku 7.5. Doprinos drugog izvora vidljiv je i u ostalim dimenzijama — dimenzija promatranja raste sa 101 na 103 kombinacije, a dimenzija vremena s 1.397 na 1.413. Jedino dimenzija datuma ostaje nepromijenjena na 12 redaka, jer oba izvora pokrivaju isto razdoblje mjerenja.

Generiranje surogat ključeva obavlja se funkcijom `row_number` nad prozorom uređenim po svim atributima dimenzije. Budući da su nakon uklanjanja duplikata svi reci međusobno različiti upravo po tim atributima, uređaj je potpun, pa je i dodjela ključeva određena: ponovno pokretanje procesa nad istim podacima dodijelit će iste ključeve istim redcima. Ključevi su cijeli brojevi koji počinju od jedinice i nemaju nikakvo značenje izvan skladišta, čime je zadovoljena osnovna svrha surogat ključa iz odjeljka 6.4.

Mehanizam sporo mijenjajućih dimenzija potreban je samo za dimenziju vizijsko-jezičnog modela. Njezina je izvedba, opisana u odjeljku 6.6.1, tipa 2: promjena inačice ne prepisuje postojeći redak nego stvara novi, uz raspon valjanosti. U kodu se taj mehanizam svodi na tri poteza. Različite se inačice prepoznaju uklanjanjem duplikata po paru modela i inačice sheme. Redni broj inačice unutar istog modela dobiva se funkcijom `row_number` nad prozorom podijeljenim po modelu i uređenim po početku valjanosti. Kraj valjanosti pojedine inačice, konačno, dobiva se funkcijom `lead`, koja u istom prozoru dohvaća početak valjanosti *sljedeće* inačice. Posljednja inačica sljedeće nema, pa joj funkcija vraća praznu vrijednost — što je upravo željeni zapis da je ta inačica još uvijek na snazi.

```
def transform_vlm_dim(vlm_df):
    get_spark_session()

    entity_window = Window.orderBy("model")
    version_window = Window.partitionBy("model").orderBy("valid_from")

```

```
    combined_df = (
        vlm_df
        .select("model", "schema_v", "prompt_v", "valid_from")
        .dropDuplicates(["model", "schema_v"])
        .withColumn("vlm_id", dense_rank().over(entity_window))
        .withColumn("version", row_number().over(version_window))
        .withColumn("date_from", col("valid_from"))
        .withColumn("date_to", lead("valid_from").over(version_window)
            )
    )

    window = Window.orderBy("model", "valid_from")
    combined_df = combined_df.withColumn("vlm_tk", row_number().over(
        window))

    return combined_df.select(
        "vlm_tk", "version", "date_from", "date_to", "vlm_id", "model"
            , "schema_v", "prompt_v"
    ).orderBy("vlm_tk")
```

Listing 6: Sporo mijenjajuća dimenzija tipa 2 za vizijsko-jezični model

Vrijedi uočiti razliku između dvaju identifikatora u tom kodu. Stupac `vlm_tk` surogat je ključ i različit je za svaki redak, jer svaki redak predstavlja jedno stanje modela u vremenu. Stupac `vlm_id` identificira *entitet* — isti model kroz sve svoje inačice — pa je za oba retka jednak. Bez njega ne bi bilo moguće postaviti pitanje o modelu neovisno o inačici. Zbog toga se dobiva funkcijom `dense_rank`, koja istoj vrijednosti modela dodjeljuje isti broj, a ne `row_number`, koja bi ih razlikovala.

Preostale dimenzije nemaju posebnosti te vrste, ali dvije zaslužuju kratku napomenu. Dimenzija uvjeta jedina uvodi izvedene vrijednosti kojih u izvoru nema: temperatura, količina oborine i šifra vremena pretvaraju se u razrede grupiranjem opisanim u odjeljku 6.6.4, pri čemu se šifra vremena preslikava u kategoriju rječnikom sastavljenim prema WMO standardu. U podacima se javlja devet različitih šifri vremena i sve su rječnikom pokrivene. Dimenzija vozila, s druge strane, jedina spaja tri šifrarnika — klase, marke i boje — kako bi dobila tekstualne nazive umjesto stranih ključeva, čime se provodi denormalizacija svojstvena zvjezdastoj shemi.

7.3.2 Transformacija tablice činjenica

Izgradnja tablice činjenica najsloženiji je korak procesa i odvija se u četiri faze. Prvo se oba izvora svode na zajednički spljošteni oblik od 24 stupca — podaci iz baze spajanjem devet od deset tablica relacijskog modela, a podaci iz datoteke preimenovanjem stupaca. Jedina tablica koja u tom spajanju ne sudjeluje jest tablica lokacije, jer njezini atributi pripadaju dimenziji kamere, a ne tablici činjenica. Tako dobivena dva okvira spajaju se u jedan. Drugo, izvode se pomoćni stupci: kalendarski datum, sat i minuta iz vremenske

oznake te broj vozila zabilježenih u istom prolasku. Treće, nad tim se okvirom primjenjuje ista funkcija grupiranja koja je stvorila razrede u dimenziji uvjeta. Četvrto, pretragom se za svaki redak pronalaze surogat ključevi svih sedam dimenzija.

Redoslijed prvih dviju faza nije proizvoljan. Broj vozila u prolasku računa se prozorskom funkcijom nad skupinom redaka s istim identifikatorom prolaska, a podjela skupa podataka bila je slučajna po recima, ne po prolascima. Posljedica je da se prolazak s više vozila mogao rascijepiti između dvaju izvora: od 902 prolaska s više od jednog vozila, njih 306 ima vozila u oba izvora. Kada bi se ta prozorska funkcija primijenila prije spajanja izvora, svi bi ti prolasci dobili premalu vrijednost. Spajanjem izvora prije računanja cjelina se obnavlja, pa raspodjela broja vozila po prolasku odgovara stvarnosti: 34.458 prolazaka s jednim vozilom, 879 s dva, 22 s tri i jedan s pet.

Treća faza — ponovna primjena grupiranja — rješava problem usklađenosti. Tablica činjenica ne može se s dimenzijom uvjeta povezati po temperaturi jer dimenzija temperaturu ne sadrži; sadrži njezin razred. Da bi se veza uspostavila, i redak činjenice mora imati taj razred, i to izveden *istim* pravilom. Zato se ne piše druga inačica pravila nego se poziva ista funkcija koja je razrede stvorila u dimenziji. Time je nemoguće da se dva pravila raziđu.

Četvrta je faza pretraga ključeva. Za četiri dimenzije — datuma, vremena, kamere i vizijsko-jezičnog modela — ona je izravna, jer se spaja po vrijednostima kojih u podacima uvijek ima. Za preostale tri dimenzije potreban je oprez.

```
def _null_safe_join(left, right, columns):
    condition = reduce(lambda a, b: a & b, [left[c].eqNullSafe(right[c
        ]) for c in columns])
    return left.join(right, condition, "left")
```

Listing 7: Spajanje otporno na prazne vrijednosti

Razlog za tu funkciju leži u naravi praznih vrijednosti u SQL-u i u Sparku. Usporedba `NULL = NULL` ne daje istinu nego nepoznatu vrijednost, pa uvjet spajanja nad stupcem koji sadrži prazne vrijednosti ne uspijeva ni kada su obje strane jednako prazne. Dimenzija vozila spaja se po šest stupaca, od kojih dva — karoserija i razdoblje — sustavno sadrže prazne vrijednosti: karoserija je popunjena samo za automobile i dostavna vozila, a razdoblje samo za zapise nastale novijom inačicom sheme. Mjereno nad konačnim skupom podataka, 21.134 od 36.287 redaka, odnosno 58,2 %, sadrži barem jednu praznu vrijednost među tih šest stupaca. Uz običnu usporedbu svi bi ti redci ostali bez ključa vozila — više od polovice tablice činjenica ne bi imalo vezu prema dimenziji koja je za analizu najvažnija. Operator `eqNullSafe` tu usporedbu mijenja tako da dvije prazne vrijednosti smatra jednakima, čime se veza uspostavlja.

Za dimenzije uvjeta i promatranja ista je zaštita primijenjena, iako u njihovim ključevima praznih vrijednosti nema — izmjereno, nula u svih osam obilježja promatranja i nula u četiri obilježja uvjeta. Ondje je riječ o mjeri opreza, a ne o nužnosti: ako bi se skup podataka proširio zapisom u kojemu neko obilježje nedostaje, spajanje bi i dalje radilo.

```
def transform_vehicle_detection_fact(tables, date_dim, time_dim,
    camera_dim,
                                     vehicle_dim, conditions_dim,
                                         vlm_dim,
                                     observation_dim, csv_df=None):
    get_spark_session()

    flat_df = _from_postgres(tables).select(*FLAT_COLUMNS)

    if csv_df:
        flat_df = flat_df.unionByName(_from_csv(csv_df).select(*
            FLAT_COLUMNS))

    flat_df = (
        flat_df
        .withColumn("full_date", to_date(col("ts")))
        .withColumn("hour", hour(col("ts")))
        .withColumn("minute", minute(col("ts")))
        .withColumn("objects_in_event", count("*").over(Window.
            partitionBy("motion_id")))
    )

    flat_df = categorize_conditions(flat_df)

    joined = (
        flat_df
        .join(date_dim, ["full_date"], "left")
        .join(time_dim, ["hour", "minute"], "left")
        .join(camera_dim.select("camera_tk", col("code").alias("
            camera_id")), ["camera_id"], "left")
        .join(vlm_dim.select("vlm_tk", "schema_v"), ["schema_v"], "
            left")
    )

    joined = _null_safe_join(
        joined,
        vehicle_dim.select("vehicle_tk", "vehicle_class", "brand", "
            model", "color", "body", "era"),
        ["vehicle_class", "brand", "model", "color", "body", "era"],
    )
    joined = _null_safe_join(
        joined,
        conditions_dim,
        ["weather_category", "temperature_bucket", "
            precipitation_bucket", "lighting"],
    )
    joined = _null_safe_join(
        joined,
        observation_dim,
        ["truncated", "plate_visible", "headlights_on", "
```

```
            commercial_markings",
        "cause", "direction", "special_use", "road_condition"],
    )

    window = Window.orderBy("motion_id", "vehicle_tk")
    joined = joined.withColumn("fact_vehicle_detection_tk", row_number
        ().over(window))

    return joined.select(
        "fact_vehicle_detection_tk", "date_tk", "time_tk", "camera_tk"
            , "vehicle_tk",
        "conditions_tk", "vlm_tk", "observation_tk", "motion_id",
        "peak_mass", "dur_ms", "objects_in_event",
    ).orderBy("fact_vehicle_detection_tk")
```

Listing 8: Izgradnja tablice činjenica

Završni odabir stupaca pokazuje koliko se od spljoštenog okvira zadržava. U njega je ušlo 24 opisna stupca iz oba izvora, a iz njega izlazi dvanaest: vlastiti surogat ključ, sedam stranih ključeva prema dimenzijama, degenerirana dimenzija `motion_id` i tri mjere. Svi ostali opisni podaci u tablici činjenica više ne postoje — oni su prešli u dimenzije, a redak činjenice do njih dolazi preko ključa.

7.4 Faza učitavanja (load)

Punjenje skladišta podataka moguće je izvesti odjednom ili zapis po zapis. Ovdje je odabrano **punjenje odjednom**: za svaku od osam tablica upućuje se jedno zapisivanje koje prenosi cijeli njezin sadržaj. Razlog je vremenska učinkovitost — zapisivanje zapis po zapis značilo bi 36.287 pojedinačnih naredbi za samu tablicu činjenica, svaku s vlastitim mrežnim odlaskom i povratkom, dok skupno zapisivanje isti posao obavlja u paketima.

```
def write_spark_df_to_postgres(spark_df: DataFrame, table_name: str,
    mode: str = "append"):
    print(f"Writing to table `{table_name}` with mode `{mode}`...")

    spark_df.write.jdbc(
        url=JDBC_URL,
        table=table_name,
        mode=mode,
        properties=CONNECTION_PROPERTIES,
    )
    print(f"Done writing to `{table_name}`.")
```

Listing 9: Zapisivanje pripremljenog okvira u skladište podataka

Dva su detalja u tom kodu bitnija nego što se čine. Prvi je način zapisivanja `append`, a ne `overwrite`. Spark prepisivanje izvodi tako da odredišnu tablicu obriše i ponovno je stvori prema shemi okvira koji zapisuje. Time bi se izgubilo sve što je definirano u

trećoj kontrolnoj točki: primarni ključevi, strani ključevi prema dimenzijama i tipovi stupaca. Odabir načina `append` znači da tablice ostaju onakve kakvima ih je stvorila skripta dimenzijskog modela, a ETL proces u njih samo dodaje retke. Cijena je te odluke u tome što ponovno pokretanje procesa nad već napunjenim skladištem retke udvostručuje, pa se prije ponovnog pokretanja shema mora isprazniti — za što u trećoj kontrolnoj točki postoji zasebna skripta.

Drugi je detalj redoslijed zapisivanja. Tablice se zapisuju onim redom kojim ih je vratila faza transformacije: prvo svih sedam dimenzija, a tek na kraju tablica činjenica. Taj redoslijed nije kozmetički. Tablica činjenica ima strane ključeve prema svim dimenzijama, pa bi njezino zapisivanje prije njih prekršilo referencijalni integritet i baza bi ga odbila.

Prema načinu izvedbe, provedeno punjenje odgovara *prvom punjenju* skladišta podataka: obuhvaća sve dostupne podatke iz oba izvora, izvodi se nad praznim skladištem i ne prati promjene u izvorišnom sustavu.

7.5 Verifikacija učitanih podataka

Kontrolna točka traži da se nakon punjenja provjeri unos podataka i utvrdi da ništa nije izgubljeno. Provjera je izvedena skriptom `verify_etl.sql`, koja se sastoji od četiri upita provjere i četiri analitička izvještaja. Prvi upit prebrojava retke u svih osam tablica skladišta, drugi uspoređuje zbroj redaka iz oba izvora s brojem redaka u tablici činjenica, treći prebrojava prazne strane ključeve, a četvrti ispisuje sadržaj sporo mijenjajuće dimenzije.

Ključna je od njih druga provjera. Ona počiva na jednostavnoj tvrdnji: budući da je zrnatost tablice činjenica jedno detektirano vozilo, a oba izvora zajedno sadrže sva detektirana vozila, broj redaka u tablici činjenica mora biti točno jednak zbroju redaka dvaju izvora. Bilo koje odstupanje značilo bi gubitak redaka u transformaciji ili njihovo umnožavanje pri spajanju — a upravo je umnožavanje najčešća pogreška pri izgradnji tablice činjenica, jer se javlja kad god dimenzija sadrži više redaka koji odgovaraju istom uvjetu spajanja.

Tablica 14: Rezultati provjere nakon ETL procesa

Provjera	Očekivanje	Rezultat
Redaka iz baze podataka	—	29.030
Redaka iz CSV datoteke	—	7.257
Redaka u tablici činjenica	zbroj oba izvora	36.287
Praznih ključeva datuma	0	0
Praznih ključeva vremena	0	0
Praznih ključeva kamere	0	0
Praznih ključeva vozila	0	0
Praznih ključeva uvjeta	0	0
Praznih ključeva modela	0	0
Praznih ključeva promatranja	0	0

Zbroj se poklapa: $29.030 + 7.257 = 36.287$. Nijedan redak nije izgubljen niti umnožen.

```
SELECT
    COUNT(*) FILTER (WHERE date_tk        IS NULL) AS null_date,
    COUNT(*) FILTER (WHERE time_tk        IS NULL) AS null_time,
    COUNT(*) FILTER (WHERE camera_tk      IS NULL) AS null_camera,
    COUNT(*) FILTER (WHERE vehicle_tk     IS NULL) AS null_vehicle,
    COUNT(*) FILTER (WHERE conditions_tk  IS NULL) AS null_conditions,
    COUNT(*) FILTER (WHERE vlm_tk         IS NULL) AS null_vlm,
    COUNT(*) FILTER (WHERE observation_tk IS NULL) AS null_observation
FROM dw.fact_vehicle_detection;
```

Listing 10: Provjera da nijedan redak nije ostao bez veze prema dimenziji

Provjera praznih stranih ključeva dopunjuje prvu. Broj se redaka mogao poklopiti i da je pretraga ključeva zakazala, jer je riječ o lijevom spajanju koje redak zadržava i onda kada mu par nije pronađen — samo bi mu ključ ostao prazan. Sedam nula u ovoj provjeri potvrđuje da je za svaki redak pronađen par u svakoj dimenziji, što je ujedno i praktična potvrda da je spajanje otporno na prazne vrijednosti nužno: bez njega bi stupac `vehicle_tk` u ovoj provjeri pokazao 21.134 prazne vrijednosti.

Četvrti upit provjere ispisuje dimenziju vizijsko-jezičnog modela i potvrđuje da je mehanizam sporo mijenjajuće dimenzije proizveo očekivani zapis: dva retka istog entiteta, prvi s rasponom valjanosti zatvorenim 27. 08. 2026. u 11:46, drugi s praznim krajem valjanosti, što označava inačicu koja je na snazi.

Preostala četiri upita skripte nisu provjere nego kratki analitički izvještaji: promet po danima s razlikovanjem radnih dana i vikenda, najčešće klase vozila s prosječnim trajanjem prolaska, promet po dijelu dana i vremenskim uvjetima te vozila posebne namjene po marki. Njihova je svrha pokazati da je model nakon punjenja upotrebljiv za analizu bez ikakve dodatne pripreme — svaki od njih spaja tablicu činjenica s jednom ili dvije

dimenzije i grupira po njihovim atributima. Sadržajna analiza tim putem provodi se u sljedećem poglavlju, u alatu za vizualizaciju.

7.6 Prednosti implementiranog ETL rješenja

Rješenje je modularno: izdvajanje, transformacija i učitavanje razdvojeni su u zasebne cjeline, a unutar transformacije svaka dimenzija ima vlastitu funkciju. Posljedica je da se izmjena jedne dimenzije ne dotiče ostalih i da se dodavanje nove dimenzije svodi na jednu novu funkciju i jedan redak u cjevovodu. Isto vrijedi i za izvore: dodavanje trećeg izvora značilo bi novu funkciju izdvajanja i po jednu dodatnu granu u transformacijama, bez promjene njihove logike.

Rješenje je ponovljivo. Surogat ključevi ne dodjeljuju se brojačem baze podataka nego prozorskom funkcijom nad potpuno uređenim skupom redaka, pa ponovno pokretanje nad istim ulazom daje istu shemu ključeva. To je svojstvo vrijedno jer omogućuje usporedbu dvaju pokretanja i ponovnu izgradnju skladišta bez promjene značenja ključeva.

Rješenje je, konačno, napisano u jednom jeziku i jednom okviru: izdvajanje, transformacija i učitavanje koriste isti podatkovni okvir, pa nema pretvorbi između alata na granicama faza. Time se izbjegava razred pogrešaka koji nastaje kada se podaci sele između različitih tehnologija, poput gubitka tipova ili neusklađenog tumačenja praznih vrijednosti.

Dvije stvari valja spomenuti kao svjesno prihvaćene granice. Prva je da se prozorske funkcije bez podjele — one koje dodjeljuju surogat ključeve — u Sparku izvršavaju u jednoj particiji, jer potpuno uređenje zahtijeva da svi reci prođu kroz isto mjesto. Pri redu veličine od nekoliko tisuća redaka po dimenziji to je bez posljedica, ali kod znatno većih dimenzija to bi bilo usko grlo i ključeve bi trebalo dodjeljivati drugačije. Druga je da proces podržava samo prvo punjenje; osvježavanje bi tražilo mehanizam otkrivanja promjena i drugačiji način zapisivanja. Nijedno od toga nije bilo potrebno za skup podataka koji je zatvoren i više se ne mijenja.

8 OLAP analiza i vizualizacija

Samo skladište podataka još ne donosi vrijednost. Ona nastaje tek kada se nad njim počnu postavljati pitanja, a upravo je to zadaća pete kontrolne točke: nad napunjenom zvjezdastom shemom izgraditi analitičke prikaze i na njima demonstrirati OLAP operacije.

8.1 Tableau kao OLAP alat

Kao alat za analizu i vizualizaciju odabran je Tableau Desktop. Veza prema skladištu uspostavljena je kao *živa* (engl. *live*), što znači da alat ne drži vlastitu kopiju podataka

nego svaki prikaz prevodi u SQL upit nad shemom `dw`. Time je svaka brojka na ekranu izravan odraz stanja u skladištu, a operacije koje korisnik izvodi mišem doista se izvršavaju u bazi.

Način na koji je model uveden u alat zaslužuje napomenu. Tablica činjenica i sedam dimenzija povezane su **odnosima** (engl. *relationships*), a ne klasičnim spajanjima. Razlika nije kozmetička: klasično spajanje unaprijed stvara jednu široku tablicu, pa mjera koja broji detekcije počinje brojati retke te šire tablice i vrijednosti se napuhuju. Kod odnosa alat sam bira koje su dimenzije za pojedini prikaz potrebne i spaja samo njih, na razini zrnatosti tablice činjenica. Zvjezdasta shema iz poglavlja 6 time je izravno upotrebljiva, bez ijednog ručno napisanog upita.

Uz atribute preuzete iz dimenzija definirano je i nekoliko vlastitih mjera.

Tablica 15: Vlastite mjere definirane u alatu

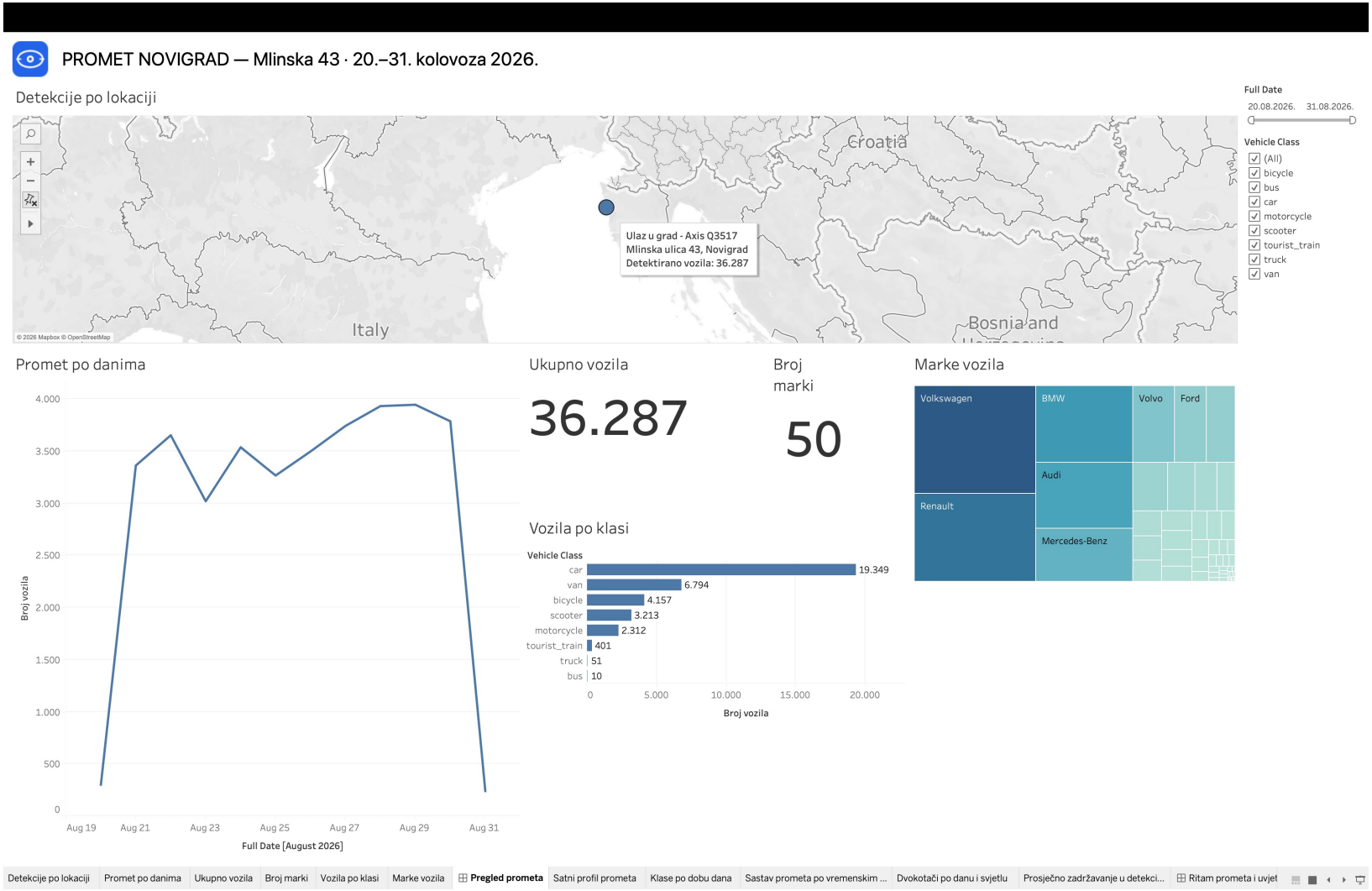
Mjera	**Definicija**
Broj vozila	`COUNT([motion_id])`
Broj marki	`COUNTD` marki uz izostavljanje vrijednosti `unknown`
Tip vozila	skupine: dvokotači, osobni automobili, gospodarska i ostala
Udio suprotnog smjera	udio detekcija sa smjerom `left` u ukupnom broju
Preklapajuće razdoblje	ograničenje na dane do 27. 08. 2026.

Mjera *Udio suprotnog smjera* pokazuje zamku svojstvenu modelu s odnosima. Prva je izvedba bila zbroj oblika `SUM(IF [direction] = 'left' THEN 1 ELSE 0 END)`, koji je davao besmislene vrijednosti jer se računao na razini dimenzije promatranja — a ona ima svega 103 retka — umjesto na razini tablice činjenica. Rješenje je da se u izrazu uvijek referencira polje iz tablice činjenica, pa konačna mjera broji `motion_id` umjesto da zbraja jedinice. Alat pritom ne javlja nikakvu pogrešku, što ovu vrstu previda čini opasnom.

Izgrađeno je ukupno šesnaest listova raspoređenih u tri ploče. Svaka ploča odgovara jednom analitičkom pitanju: prva daje opći pregled prometa, druga njegov ritam kroz dan i ovisnost o uvjetima, a treća se bavi anomalijama i kvalitetom podataka.

8.2 Ploča 1 — Pregled prometa: operacije ROLL-UP, DRILL-DOWN i SLICE

Prva ploča odgovara na pitanje što je uopće izmjereno. Sadrži kartu mjerne lokacije, kretanje broja vozila po danima, dva zbirna pokazatelja te raščlambu po klasama i markama.



Slika 6: Ploča 1 — Pregled prometa

Roll-up je vidljiv u dvama pokazateljima na sredini ploče. Vrijednosti 36.287 vozila i 50 marki najviša su moguća razina sažimanja — cijela tablica činjenica svedena na jedan broj, bez ijedne dimenzije. Zanimljivo je da mjera *Broj marki* pokazuje 50, a ne 51, jer iz prebrojavanja izostavlja vrijednost `unknown`: ta vrijednost jest redak šifrarnika, ali nije marka.

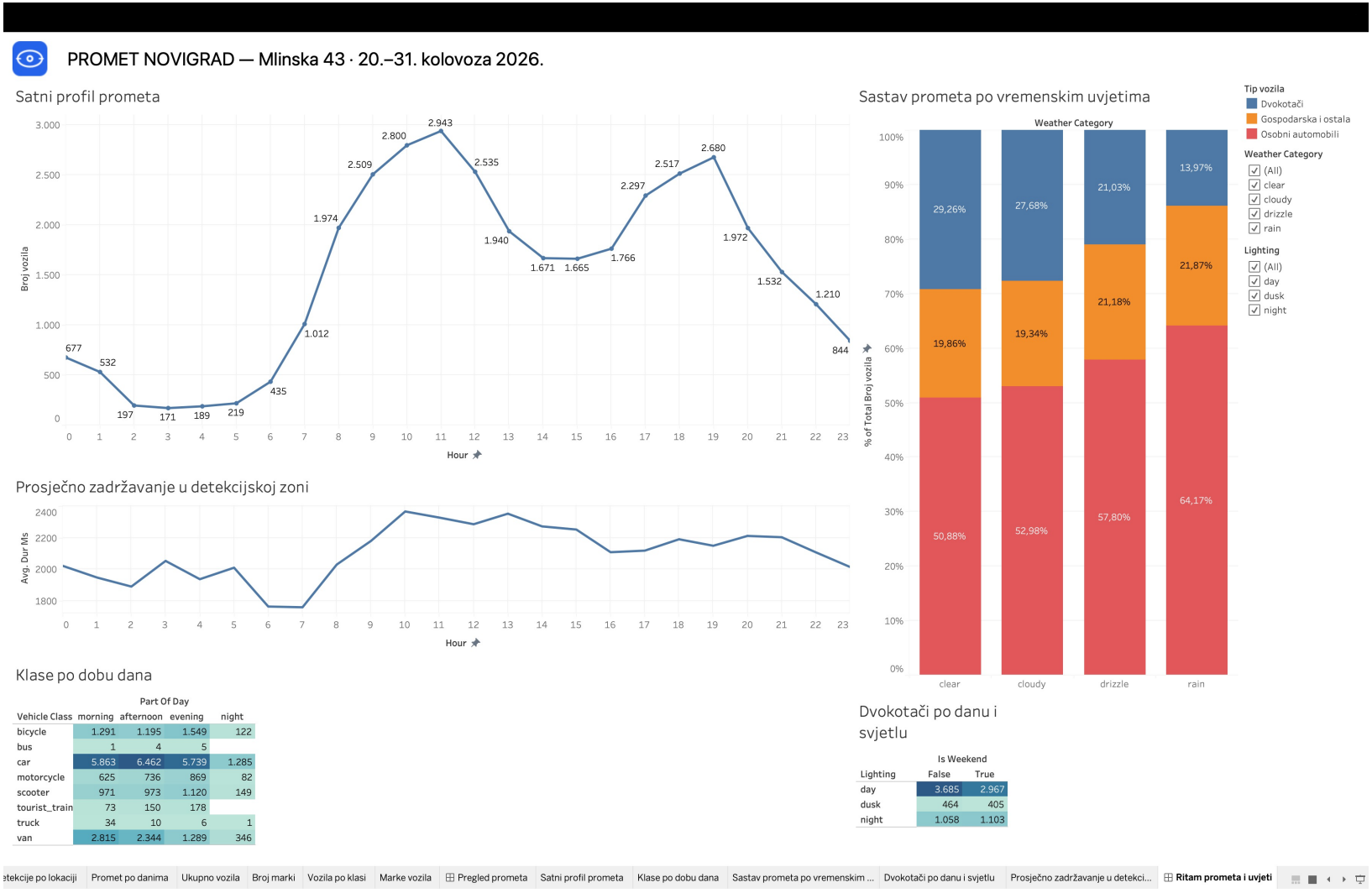
Drill-down slijedi hijerarhiju *Vozilo* definiranu u dimenziji vozila, s razinama klasa → marka → model. Panel *Vozila po klasi* prikazuje prvu razinu, a *Marke vozila* drugu. Prijelaz s jedne na drugu spuštanje je razine detalja uz nepromijenjenu mjeru, što je definicija te operacije. Drugi oblik istog spuštanja pokazuje karta: sama po sebi prikazuje jednu točku s ukupnim brojem detekcija, a dodavanjem klase vozila kao razlikovnog obilježja ista se točka razlaže na sastavnice.

Slice se izvodi filtrima uz desni rub ploče. Ograničavanje klase vozila ili raspona datuma rez je kroz kocku po jednoj dimenziji: razina detalja ostaje nepromijenjena, mijenja se samo obuhvat. Isključe li se dvokotači, od 36.287 detekcija ostaje njih 26.605 — ista mjera nad užim rezom.

Opseg djelovanja filtra pritom nije sporedna postavka. Filtar vezan samo uz jedan panel ostavlja ostale prikaze na ploči da i dalje govore o cijelom skupu, pa ploča istodobno pokazuje dva različita obuhvata i čitatelj ne može znati na što se koja brojka odnosi. Suprotna je krajnost jednako nezgodna: filtar proširen na sve listove koji dijele isti izvor podataka mijenja i preostale dvije ploče, uključujući panel na kojemu se demonstrira kockanje. Ispravan je opseg onaj između — filtar koji djeluje na prikaze jedne ploče i ne izlazi izvan nje. Filtri klase vozila i raspona datuma na ovoj su ploči postavljeni upravo tako: mijenjaju svih šest njezinih prikaza istodobno, pa zbirni pokazatelj i raščlamba po klasama uvijek govore o istom podskupu.

8.3 Ploča 2 — Ritam prometa i uvjeti: operacije DICE i PIVOT

Druga ploča spušta analizu s razine dana na razinu sata i uvodi vremenske uvjete kao dimenziju usporedbe.

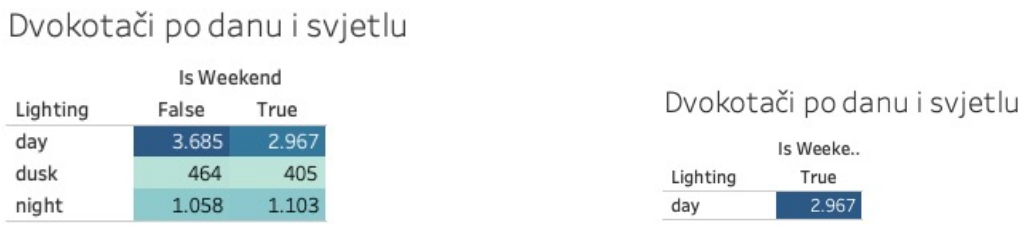


Slika 7: Ploča 2 — Ritam prometa i uvjeti

Prijelaz s prve ploče na drugu sam je po sebi **drill-down** po vremenskoj hijerarhiji: *Promet po danima* prikazuje dane, *Satni profil prometa* sate istog razdoblja. Profil pokazuje dva vrhunca, jutarnji oko 11 sati s 2.943 vozila i večernji oko 19 sati s 2.680 vozila, te noćni minimum od 171 vozila u tri sata ujutro.

Pivot je vidljiv u unakrsnoj tablici *Klase po dobu dana*. Klase vozila stoje u recima, doba dana u stupcima, a zamjenom tih dvaju položaja dobiva se isti sadržaj u drugoj orijentaciji. Operacija ne mijenja ni obuhvat ni razinu detalja — mijenja samo os po kojoj se čita — pa je korisna kada isto pitanje treba pogledati iz drugog kuta.

Dice je demonstriran na panelu *Dvokotači po danu i svjetlu*, i to je jedina operacija te vrste na cijelom projektu. Razlika u odnosu na rez leži u broju istodobno ograničenih dimenzija: rez ograničava jednu dimenziju, a kockanje dvije ili više, čime od polazne kocke ostaje manja kocka. Polazno stanje prikazuje mrežu od šest ćelija: tri vrijednosti osvjetljenja naspram dviju vrijednosti obilježja vikenda, uz klasu vozila već ograničenu na tri od osam klasa. Dodavanjem filtra na vikend i na osvjetljenje ostaje jedna ćelija s 2.967 detekcija. Tri su dimenzije ograničene istodobno — klasa na tri od osam vrijednosti, dan u tjednu na jednu od dvije, osvjetljenje na jednu od tri — dok je razina detalja ostala nepromijenjena, jer se i dalje broje pojedinačne detekcije. Upravo ta nepromijenjena razina detalja razlikuje kockanje od sažimanja.



(a) polazna kocka, šest ćelija

(b) nakon kockanja, jedna ćelija

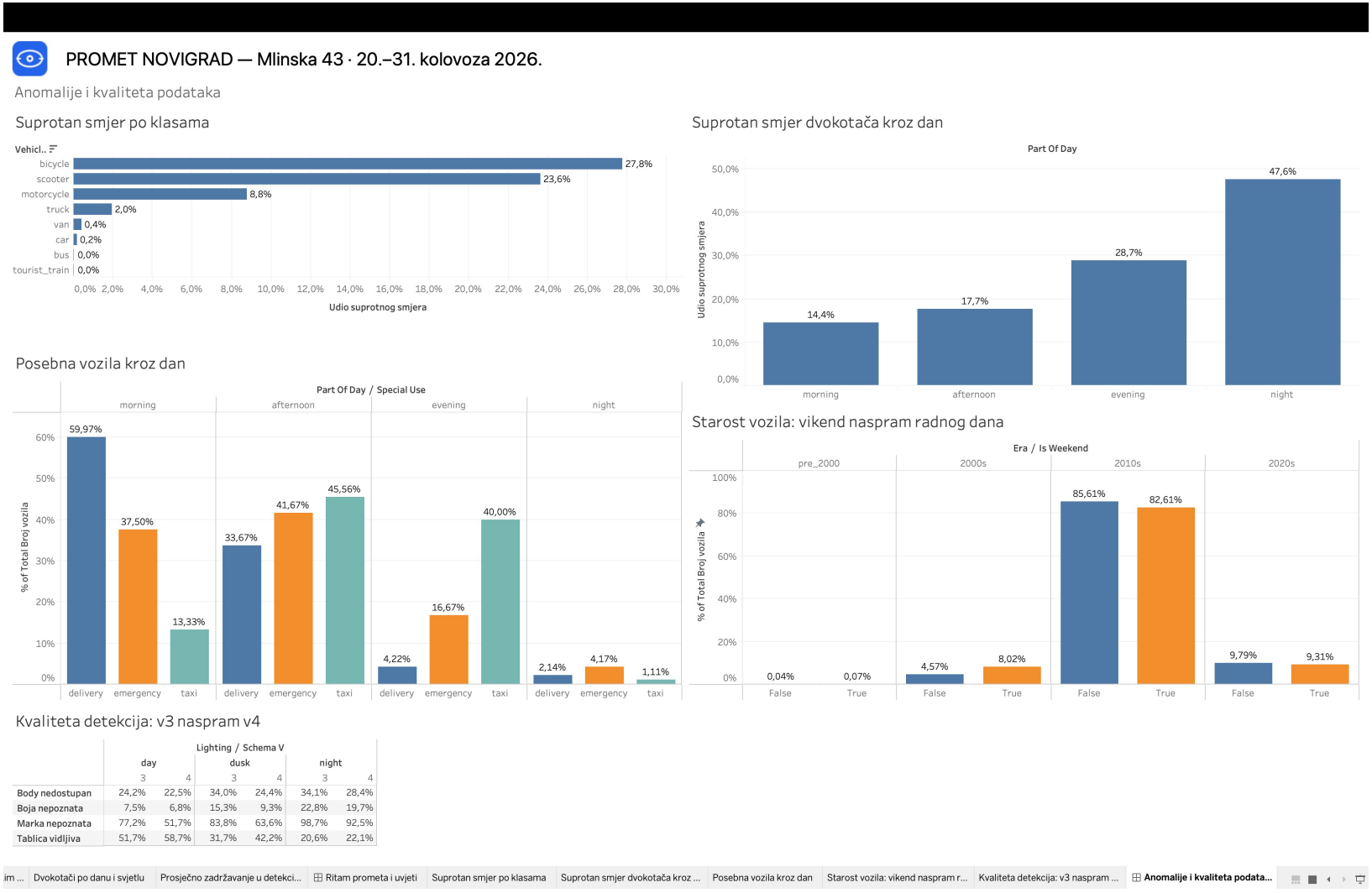
Slika 8: Operacija kockanja: ograničavanjem dana u tjednu i osvjetljenja od polazne kocke ostaje jedna ćelija

Sadržajno, najzanimljiviji je nalaz ove ploče onaj o utjecaju kiše. Panel *Sastav prometa po vremenskim uvjetima* pokazuje da kiša mijenja sastav prometa mnogo više nego njegovu količinu: udio dvokotača pada s 29,26 % po vedrom vremenu na 13,97 % po kiši, a taj udio preuzimaju osobni automobili, koji rastu s 50,88 % na 64,17 %.

Ljudi, dakle, po kiši ne ostaju kod kuće nego mijenjaju prijevozno sredstvo. Objašnjenje koje se nameće u skladu je s mjernom lokacijom: riječ je o ulazu u priobalni gradić u vrhuncu turističke sezone, gdje kiša kretanje ne zaustavlja nego ga preusmjerava. Umjesto na plažu, ide se u grad, a put koji bi se po vedrom vremenu prešao biciklom ili romobilom po kiši se prelazi automobilom. Podaci tu tvrdnju ne dokazuju izravno — oni bilježe zamjenu prijevoznog sredstva, a ne odredište putovanja — ali su s njom u skladu.

8.4 Ploča 3 — Anomalije i kvaliteta podataka: operacije SLICE, PIVOT i DRILL-DOWN

Treća ploča okrenuta je onome što u podacima odstupa od očekivanog, uključujući i kvalitetu samih anotacija.



Slika 9: Ploča 3 — Anomalije i kvaliteta podataka

Prvi je nalaz ponašanje u suprotnom smjeru na jednosmjernoj ulici. Panel *Suprotan smjer po klasama* pokazuje raspon koji se ne može pripisati pogrešci prepoznavanja: bicikli se u 27,8 % slučajeva kreću suprotno od dopuštenog smjera, romobili u 23,6 %, motocikli u 8,8 %, dok su osobni automobili na 0,2 %, a autobusi i turistički vlak na nuli. Razlika između 27,8 i 0,2 posto nije mjerni šum nego razlika u ponašanju sudionika.

Taj se nalaz zatim produbljuje. Panel *Suprotan smjer dvokotača kroz dan* primjenjuje **slice** po tipu vozila — zadržava samo dvokotače — i unutar tog reza spušta se na doba dana. Udio suprotnog smjera raste s 14,4 % ujutro na 17,7 % poslijepodne, 28,7 % navečer i 47,6 % noću. Klasa vozila pritom je držana nepromijenjenom, pa je porast posljedica doba dana, a ne promjene sastava prometa.

Pivot se javlja u dvama panelima s ugniježđenim osima: *Posebna vozila kroz dan*, gdje se doba dana i namjena vozila mogu zamijeniti, te *Starost vozila: vikend naspram radnog dana*, gdje istu ulogu imaju razdoblje proizvodnje i obilježje vikenda.

Najvažniji je prikaz na ovoj ploči ipak posljednji. Panel *Kvaliteta detekcija: v3 naspram v4* uspoređuje dvije inačice vizijsko-jezičnog modela na preklapajućem razdoblju, dakle na danima u kojima su obje bile u upotrebi, po četirima pokazateljima kvalitete anotacije. Promatra li se samo zbirna vrijednost, novija inačica djeluje mješovito: bolja je po dva pokazatelja — rjeđe joj nedostaje karoserija (24,5 % naspram 25,6 %) i rjeđe ne prepoznaje marku (65,6 % naspram 79,1 %) — ali lošija po druga dva, jer češće ne prepoznaje boju (11,1 % naspram 9,1 %) i rjeđe uočava registarsku tablicu (45,8 % naspram 48,1 %). Spuštanjem razine po osvjetljenju — što je **drill-down** — slika se potpuno mijenja. Novija je inačica bolja u *svih dvanaest* usporedbi, po sva četiri pokazatelja i u sva tri uvjeta osvjetljenja. Marku vozila po danu prepoznaje na 48,3 % vozila naspram 22,8 % kod starije inačice.

Kvaliteta detekcija: v3 naspram v4

	Schema V	
	3	4
Body nedostupan	25,6%	24,5%
Boja nepoznata	9,1%	11,1%
Marka nepoznata	79,1%	65,6%
Tablica vidljiva	48,1%	45,8%

(a) zbirni prikaz: novija inačica bolja je po dva pokazatelja, lošija po druga dva

Kvaliteta detekcija: v3 naspram v4

	Lighting / Schema V					
	day		dusk		night	
	3	4	3	4	3	4
Body nedostupan	24,2%	22,5%	34,0%	24,4%	34,1%	28,4%
Boja nepoznata	7,5%	6,8%	15,3%	9,3%	22,8%	19,7%
Marka nepoznata	77,2%	51,7%	83,8%	63,6%	98,7%	92,5%
Tablica vidljiva	51,7%	58,7%	31,7%	42,2%	20,6%	22,1%

(b) nakon spuštanja razine po osvjetljenju: novija inačica bolja je u svih dvanaest usporedbi

Slika 10: Operacija spuštanja razine otkriva Simpsonov paradoks u usporedbi dviju inačica modela

Uzrok prividne kontradikcije jest neujednačen sastav dvaju uzoraka. Novija je inačica naknadno ponovno obradila ciljani podskup zapisa u kojemu su noćne snimke bile znatno zastupljenije: 31,6 % njezinih zapisa u preklapajućem razdoblju otpada na noć, naspram 6,6 % kod starije inačice. Kako su noću svi pokazatelji lošiji, taj neuravnoteženi sastav u zbroju povlači novu inačicu prema dolje iako je u svakoj pojedinoj skupini bolja.

Riječ je o *Simpsonovu paradoksu*, pojavi u kojoj se odnos vidljiv u pojedinim skupinama obrće kada se skupine zbroje. Za rad o skladištenju podataka taj je primjer vrjedniji od bilo kojeg nalaza o prometu jer pokazuje razlog zbog kojega OLAP uopće postoji: zbirna vrijednost može navesti na pogrešan zaključak, a operacija spuštanja razine otkriva stvarni odnos. Isti je nalaz ujedno i naknadno opravdanje za odluku iz poglavlja 6 da se inačica modela vodi kao sporo mijenjajuća dimenzija: bez zasebnog retka za svaku inačicu ova usporedba ne bi bila moguća.

8.5 Zbirni pregled OLAP operacija

Tablica 16 povezuje svaki od šesnaest listova s operacijom koju prikazuje.

Tablica 16: Prikaz OLAP operacija po listovima

List	Operacija	Objašnjenje
Ploča 1 — Pregled prometa		
Ukupno vozila	roll-up	potpuno sažimanje na jednu vrijednost
Broj marki	roll-up	sažimanje uz izostavljanje `unknown`
Detekcije po lokaciji	drill-down	karta → razlaganje po klasi
Promet po danima	slice	ograničenje razdoblja i klase
Vozila po klasi	drill-down	1. razina hijerarhije *Vozilo*
Marke vozila	drill-down	2. razina: klasa → marka
Ploča 2 — Ritam prometa i uvjeti		
Satni profil prometa	drill-down	dan → sat
Prosječno zadržavanje	slice	rez po uvjetima uz drugu mjeru
Sastav prometa po uvjetima	slice	rez po kategoriji vremena
Klase po dobu dana	pivot	zamjena redaka i stupaca
Dvokotači po danu i svjetlu	**dice**	tri dimenzije ograničene istodobno
Ploča 3 — Anomalije i kvaliteta podataka		
Suprotan smjer po klasama	roll-up	udio po klasama, bez daljnje podjele
Suprotan smjer kroz dan	slice	samo dvokotači, po dobu dana
Posebna vozila kroz dan	pivot	doba dana × namjena
Starost vozila	pivot	razdoblje × vikend
Kvaliteta v3 naspram v4	**drill-down**	agregat → po osvjetljenju

Dvije operacije zaslužuju izdvajanje jer se na njima najbolje vidi razlika između prikaza i analize. Kockanje na panelu *Dvokotači po danu i svjetlu* jedina je operacija te vrste u projektu i namjerno je izvedena kao postupak, a ne kao stanje: ploča ostaje u punom prikazu od šest ćelija, a kockanje se demonstrira sužavanjem na jednu. Spuštanje razine na panelu *Kvaliteta detekcija* jedina je operacija koja mijenja zaključak, a ne samo razinu detalja.

Preostale operacije nisu proglašene složenijima nego što jesu. Panel filtriran po jednoj dimenziji rez je, a ne kockanje, bez obzira na to koliko vrijednosti ta dimenzija ima; sažimanje po atributu podizanje je razine, a ne rezanje. Ta je razlika ovdje dosljedno poštovana.

Pri razvrstavanju je pritom bitno razlikovati ograničavanje dimenzije od njezina prika-

zivanja. Rez i kockanje odnose se isključivo na dimenzije koje su filtrom ograničene na podskup vrijednosti; dimenzija razastrta po osi prikaza nije ograničena, nego prikazana u cijelosti, i ne ulazi u tu računicu. Zato je, primjerice, panel *Suprotan smjer dvokotača kroz dan* rez, a ne kockanje: filtrom je ograničen samo tip vozila, dok su sva četiri dijela dana prikazana.

9 Rasprava i ograničenja

Rezultati prikazani u prethodnim poglavljima počivaju na skupu podataka koji je u cijelosti prikupljen za potrebe ovog rada. To je njegova najveća prednost, ali i izvor ograničenja koja treba navesti jer utječu na to koliko se nalazi smiju poopćiti.

9.1 Ograničenja mjerne postavke

Sustav je **jednokameran**. Sve 36.287 detekcija dolazi s jedne kamere na jednoj lokaciji, pa dimenzija kamere u skladištu ima točno jedan redak. Nalazi vrijede za tu prometnicu i ne mogu se bez provjere prenijeti na grad u cjelini. Za usporedbu lokacija trebalo bi postaviti drugu kameru, što model podržava — dimenzija kamere postoji upravo zato — ali podaci za to ne postoje.

Položaj kamere određuje što se uopće može vidjeti. Kamera gleda cestu pod kutom s jedne strane, pa su vozila u bližem traku vidljiva bolje od onih u daljem, a veće vozilo može zakloniti manje koje se kreće pored njega. Iz istog razloga registarska tablica nije uvijek u vidnom polju. To nije kvar postavke nego njezino svojstvo, ali znači da broj detekcija treba čitati kao broj vozila *koje je kamera vidjela*, a ne kao potpun broj vozila koja su prošla.

Broj sličica u sekundi nije bio stalan. Kamera je nominalno isporučivala 25 sličica u sekundi, ali je u razdobljima slabije mrežne veze stvarna stopa padala znatno ispod toga. Kada između dviju sličica prođe više vremena, brzo vozilo može proći kroz zonu detekcije a da u njoj ne bude zabilježeno ni na jednoj sličici. Takvi propušteni prolasci ne ostavljaju trag u podacima — ne postoji zapis o vozilu koje nije viđeno — pa se njihov broj ne može ni procijeniti iz samog skupa. Posljedica je da apsolutne brojke treba smatrati donjom granicom stvarnog prometa, dok su odnosi među kategorijama, na kojima počiva većina nalaza, znatno otporniji na taj gubitak.

9.2 Pouzdanost anotacija vizijsko-jezičnog modela

Sadržaj svake detekcije određuje vizijsko-jezični model, a njegova točnost nije jednaka za sve atribute. Provjera nad ručno označenim uzorkom, prikazana u odjeljku 3.9, pokazala je raspon od 87 % za marku vozila do 39 % za smjer kretanja. Atributi na vrhu tog raspona

mogu se koristiti bez ograde; oni pri dnu ne mogu.

Smjer kretanja bio je toliko nepouzdan da izvorne vrijednosti modela u konačnom skupu podataka nisu zadržane, nego su zamijenjene presudama ručne revizije. Nalaz o kretanju u suprotnom smjeru iz odjeljka 8.4 zato ne počiva na izlazu modela nego na provjerenim vrijednostima. Slično je i s osvjetljenjem, koje je u 22,2 % redaka ispravljeno izračunom iz položaja sunca umjesto da se preuzme modelova procjena.

Preostaje ograničenje koje se ne da ukloniti: udio nepoznatih vrijednosti. Marka vozila nije prepoznata u 71,6 % detekcija, a razdoblje proizvodnje uopće ne postoji u zapisima nastalim starijom inačicom sheme. Analize koje se oslanjaju na te atribute zato govore o *prepoznatom* dijelu prometa, a ne o cjelini.

9.3 Nejednolika pokrivenost uzorka

Mjerenje obuhvaća dvanaest kalendarskih dana, od 20. do 31. kolovoza 2026. Prvi i posljednji dan nisu potpuni — prvog je snimanje počelo navečer, posljednjeg je prekinuto ujutro — pa na njih otpada 300, odnosno 239 detekcija, naspram približno 3.500 na svaki od deset punih dana. U analizama po danima ta dva dana treba čitati kao rubna.

Razdoblje je uz to kratko i sezonski usko. Riječ je o drugoj polovici kolovoza u priobalnom gradiću, dakle o vrhuncu turističke sezone. Sastav prometa u tom razdoblju gotovo sigurno nije jednak onome u studenom, pa se nalazi o udjelu dvokotača ili o ponašanju po kiši odnose na promatrano razdoblje, a ne na godinu. Dvanaest dana također ne dopušta tvrdnje o tjednim ili mjesečnim trendovima — razlika između radnog dana i vikenda može se promatrati, ali samo na dva vikenda.

Nejednolikost postoji i unutar samog uzorka. Kao što je pokazano u odjeljku 8.4, dvije inačice vizijsko-jezičnog modela nisu obradile usporediv presjek podataka: novija je naknadno ponovno obradila podskup s izrazito većim udjelom noćnih snimaka. Svaka usporedba tih dviju inačica zato mora kontrolirati uvjete osvjetljenja, inače daje obrnut zaključak.

9.4 Primjena rezultata u produkcijskom sustavu

Sustav za prikupljanje opisan u poglavlju 3 nije izgrađen isključivo za potrebe ovog rada. On je dio postojećeg produkcijskog sustava za brojanje prometa, koji podatke o prolascima prikuplja i prikazuje, ali nema sloj skladišta podataka ni podršku za analitičku obradu. Drugim riječima, brojke u njemu postoje, ali se ne mogu promatrati po dimenzijama, spuštati niz hijerarhiju ni uspoređivati kroz vrijeme onako kako to omogućuje dimenzijski model. Upravo tu prazninu ovaj rad popunjava, pa je najizravnija primjena rezultata proširenje takvog sustava analitičkim slojem. Prikupljanje se pritom ne mora zaustaviti završetkom rada, čime se otvara nekoliko smjerova.

Najizravniji je **produljenje mjerenja**. Skladište je već izvedeno tako da prihvaća nove

podatke: dimenzija datuma nije ograničena na promatrano razdoblje, a punjenje bi umjesto prvog punjenja prešlo u periodičko osvježavanje. Time bi otpalo ograničenje sezonske uskosti iz prethodnog odjeljka.

Drugi je smjer **proširenje na više lokacija**. Model je za to spreman jer dimenzija kamere postoji i nosi podatke o lokaciji; dodavanjem druge kamere ista bi analiza postala usporedba prometnica, a otvorila bi se i operacija unakrsnog pregleda koja s jednom lokacijom nema smisla.

Treći je smjer **praktična upotreba nalaza**. Podatak da udio kretanja u suprotnom smjeru kod dvokotača noću doseže 47,6 % ima izravnu vrijednost za prometnu sigurnost, jednako kao i satni profil prometa za planiranje prometnih režima. Skladište u tom obliku može poslužiti kao podloga za takve odluke, uz ograde navedene u ovom poglavlju.

10 Zaključak

U radu je izgrađeno skladište podataka za analizu cestovnog prometa, i to nad skupom podataka koji je u cijelosti prikupljen vlastitim sustavom. Lanac je proveden do kraja: od kamere i prepoznavanja prolaska vozila, preko relacijskog modela s deset tablica, do zvjezdaste sheme sa sedam dimenzija i 36.287 redaka u tablici činjenica, koju ETL proces puni iz dvaju izvora bez ijednog izgubljenog ili umnoženog retka.

Odluka da se skup podataka izgradi umjesto preuzme pokazala se opravdanom na dva mjesta. Prvo, omogućila je mjerenje pouzdanosti pojedinih obilježja nad ručno provjerenim uzorkom, pa se u radu točno zna koje se obilježje smije koristiti bez ograde, a koje ne. Drugo, promjena inačice modela za označavanje tijekom mjerenja dala je stvaran primjer sporo mijenjajuće dimenzije — ne izmišljen radi zadatka, nego zabilježen jer se dogodio.

Analiza je pokazala i nekoliko nalaza koje sam broj vozila ne bi otkrio. Kiša mijenja sastav prometa mnogo više nego njegovu količinu: udio dvokotača pada s 29,26 % na 13,97 %, a taj udio preuzimaju osobni automobili. Udio kretanja u suprotnom smjeru na jednosmjernoj ulici kod bicikala doseže 27,8 %, naspram 0,2 % kod osobnih automobila, a kod dvokotača noću raste do 47,6 %.

Najzanimljiviji je nalaz ipak onaj koji objašnjava zašto OLAP uopće postoji. Usporedba dviju inačica modela za označavanje u zbirnom prikazu daje mješovit rezultat, a spuštanjem razine po osvjetljenju novija inačica pobjeđuje u svih dvanaest usporedbi. Riječ je o Simpsonovu paradoksu: agregat navodi na pogrešan zaključak, a operacija spuštanja razine otkriva stvarni odnos.

Ograničenja rada navedena su u poglavlju 9 i uglavnom proizlaze iz opsega mjerenja — jedna kamera, dvanaest dana i jedna sezona. Model je izveden tako da ta ograničenja može nadrasti: dimenzija kamere spremna je za drugu lokaciju, a punjenje skladišta za prelazak s prvog punjenja na periodičko osvježavanje. Time bi analitički sloj izgrađen u ovom radu mogao postati trajan dio postojećeg sustava za brojanje prometa, koji podatke

danas prikuplja, ali ih ne analizira.

Literatura

[1] GStreamer Project. *GStreamer: Open Source Multimedia Framework*. `https://gstreamer.freedesktop.org/`. 2026.

[2] Qwen Team. *Qwen3-VL: Vision-Language Models*. `https://github.com/QwenLM/Qwen3-VL`. 2025.

[3] Open-Meteo. *Open-Meteo: Free Weather API*. `https://open-meteo.com/`. 2026.

[4] Ralph Kimball i Margy Ross. *The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling*. 3. izdanje. Indianapolis: John Wiley & Sons, 2013.

[5] Matei Zaharia i dr. "Apache Spark: A Unified Engine for Big Data Processing". *Communications of the ACM* 59.11 (2016.), str. 56–65.

[6] William H. Inmon. *Building the Data Warehouse*. 4. izdanje. Indianapolis: John Wiley & Sons, 2005.

[7] Surajit Chaudhuri i Umeshwar Dayal. "An Overview of Data Warehousing and OLAP Technology". *ACM SIGMOD Record* 26.1 (1997.), str. 65–74.

[8] Jim Gray i dr. "Data Cube: A Relational Aggregation Operator Generalizing Group-By, Cross-Tab, and Sub-Totals". *Data Mining and Knowledge Discovery* 1.1 (1997.), str. 29–53.

[9] PostgreSQL Global Development Group. *PostgreSQL Documentation*. `https://www.postgresql.org/docs/`. 2026.

[10] Salesforce. *Tableau*. `https://www.tableau.com/`. 2026.

Popis slika

1 Arhitektura sustava za prikupljanje podataka 5

2 Konceptualni model podataka 21

3 Prošireni model podataka s tipovima i ograničenjima 23

4 Zvjezdasta shema skladišta podataka 34

5 Tijek ETL procesa: dva izvora, pripremno područje i skladište podataka . 37

6 Ploča 1 — Pregled prometa 50

7 Ploča 2 — Ritam prometa i uvjeti 52

8 Operacija kockanja: ograničavanjem dana u tjednu i osvjetljenja od polazne kocke ostaje jedna ćelija 53

9 Ploča 3 — Anomalije i kvaliteta podataka 55

10 Operacija spuštanja razine otkriva Simpsonov paradoks u usporedbi dviju inačica modela 57

Popis tablica

1 Kontrolne točke, pripadajuća poglavlja i programska izvedba 3
2 Elementi cjevovoda za prikupljanje podataka 7
3 Izmjerena točnost anotacije na slijepom uzorku od 150 detekcija 11
4 Atributi skupa podataka po skupinama 13
5 Broj detekcija po danima 14
6 Raspodjela detekcija po klasi vozila 15
7 Razdioba mjera događaja 15
8 Udio nepoznatih i praznih vrijednosti po atributu 16
9 Broj redaka po tablici relacijskog modela 26
10 Dimenzije zvjezdaste sheme 30
11 Hijerarhije u dimenzijskim tablicama 30
12 Sadržaj dimenzije `dim_vlm` 31
13 Koraci cjevovoda transformacije i izvori iz kojih se pune 40
14 Rezultati provjere nakon ETL procesa 47
15 Vlastite mjere definirane u alatu 49
16 Prikaz OLAP operacija po listovima 58